

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»

**И. Ф. Астахова, Е. В. Трофименко,
Ю. В. Хицкова, Г. В. Горковенко**

**ПРОЕКТИРОВАНИЕ
ИНФОРМАЦИОННЫХ СИСТЕМ.
СТРУКТУРНОЕ ПРОЕКТИРОВАНИЕ**

Учебное пособие

Воронеж
Издательский дом ВГУ
2024

УДК 004.652.5
ББК 32.972.1
А91

Р е ц е н з е н т –
доктор физико-математических наук,
профессор *М. А. Артемов*

Астахова И. Ф.

А91 Проектирование информационных систем. Структурное проектирование : учебное пособие / И. Ф. Астахова, Е. В. Трофименко, Ю. В. Хицкова, Г. В. Горковенко ; Воронежский государственный университет. – Воронеж : Издательский дом ВГУ, 2024 – 80 с.

Учебное пособие содержит описание языка для проектирования информационных систем и подборку примеров, задач и упражнений различной степени сложности для обеспечения лабораторных занятий по изучению этих возможностей в рамках учебного курса «Проектирование ИС», посвященного информационным системам.

Для студентов, обучающихся по направлению подготовки 02.04.02 «Фундаментальная информатика и информационные технологии» и направлению «Математическое обеспечение и администрирование информационных систем» по дисциплине Б1.В.06 «Проектирование информационных систем».

УДК 004.652.5
ББК 32.972.1

© Астахова И. Ф., Трофименко Е. В., Хицкова Ю. В., Горковенко Г. В., 2024

© Воронежский государственный университет, 2024

© Оформление. Издательский дом ВГУ, 2024

Содержание

Введение	4
1. СЛОЖНОСТЬ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	5
1.1. Основные понятия и определения	5
1.2. Сложность информационных систем	7
1.3. Модели жизненного цикла информационной системы	11
2 СТРУКТУРНОЕ (ФУНКЦИОНАЛЬНОЕ) ПРОЕКТИРОВАНИЕ	25
2.1. Методология построения процессов IDEF0	27
2.2. Методология IDEF3	53
2.3. Методология DFD	64
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	77
ЛИТЕРАТУРА	79

ВВЕДЕНИЕ

Под проектированием информационных систем в широком смысле понимается процесс преобразования входной информации об объекте проектирования, о методах проектирования и об опыте проектирования объектов аналогичного назначения в соответствии с ГОСТом в проект ИС (действующую ИС и проектно-технологическую документацию). В узком смысле – это вторая стадия ЖЦ ИС. Технология проектирования ИС – это совокупность методологии и средств проектирования ИС, а также методов и средств организации проектирования (управления процессом создания и модернизации проекта ИС). Рассматривается технология структурного. Структурное проектирование рассматривается с помощью DEF). DEF3 и DFD.

Проектирование IDEF0, IDEF3, DFD рассматриваются с помощью ERWIN или BPWIN. Хочется поблагодарить студентов 3 курса 9 и 91 группы и 3 курса ФИИТ, выполнивших необходимые лабораторные задачи: Кравцову Маргариту и Худякова Александра, Алехину Екатерину и Маслову Анастасию, Певцову Ангелину.

1 СЛОЖНОСТЬ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

При решении определенной практической задачи программирования информационной системы в реальной жизни приходится сталкиваться с необходимостью проектирования информационных систем, построенных на базе компьютерной техники, предназначенных для хранения, поиска, обработки и передачи больших объемов информации. Таких задач очень много, например, создание информационно-поисковой системы традиционной книжной библиотеки или проектирование системы автоматического управления производством.

Особое место занимают современные информационные технологии ведения электронной коммерции, работа с заказчиками и поставщиками. И в этом направлении проектирование и развитие информационных систем не возможно без знания основных методологий и программных средств, позволяющих в кратчайшие сроки и без ошибок управлять этими процессами. Компании получают колоссальные конкурентные преимущества, если уровень развития информационных систем соответствует уровню развития предприятия. Улучшается инвестиционная привлекательность компании, основные бизнес-процессы становятся прозрачными и понятными для контроля и управления, исключаются ошибки, брак и связанные с ними потери и времени, и средств, в конечном счете, увеличивается прибыль компании.

1.1. Основные понятия и определения

Основные понятия в последние годы не претерпели сильных изменений, формулировки стали более точными и лаконичными, исключаями неоднозначность понятий. Наиболее полные определения представлены в Федеральных законах Российской Федерации и стандартах.

Информация – «сведения (сообщения, данные) независимо от формы их представления» [1].

Информационные технологии – «процессы, методы поиска, сбора, хранения, обработки, предоставления, распространения информации и способы осуществления таких процессов и методов» [1].

Информационная система – «совокупность содержащейся в базах данных информации и обеспечивающих ее обработку информационных технологий и технических средств» [1].

Проектирование информационных систем – это упорядоченная совокупность методологий и средств создания или модернизации информационных систем.

Управление информационными системами – «применение методов управления процессами планирования, анализа, дизайна, создания, внедрения и эксплуатации информационной системы организации для достижения ее целей» [2].

Жизненный цикл информационных системы – «развитие рассматриваемой системы во времени, начиная от замысла и кончая списанием» [3]. Модель жизненного цикла – «структурная основа процессов и действий, относящиеся к жизненному циклу, которая служит в качестве общей ссылки для установления связей и взаимопонимания сторон» [3].

Архитектура информационных систем – это концепция, определяющая модель, структуру, выполняемые функции и взаимосвязь компонентов информационной системы.

Бизнес-процесс – это цепочка взаимосвязанных действий, направленных на создание товарной продукции или услуги. Регламент бизнес-процесса – это четко определенный порядок выполнения бизнес-процесса, определяющий состав и действия участников.

Модель данных – это система организации данных и управления ими.

Методология проектирования информационных систем – это совокупность принципов проектирования (моделирования), выраженная в определенной концепции.

Средства моделирования – это программы описания и моделирования систем.

Типовое проектное решение (ТПР) – это многократно используемое проектное решение.

Нотации – это определенные способы представления элементов информационной системы.

Реинжиниринг бизнес-процессов – это фундаментальная реорганизация бизнес-процессов с целью повышения их эффективности.

Системный подход – процесс рассмотрения любой системы в качестве совокупности взаимосвязанных элементов.

Процессный подход – представление любой системы в качестве совокупности процессов.

Функциональный подход – предусматривает четкое закрепление за каждой структурной единицей набора функций.

Техническое задание – документ, используемый заказчиком в качестве средства для описания и определения задач, выполняемых при реализации договора [4].

1.2. Сложность информационных систем

Промышленные программные продукты всегда обладают некоторым уровнем сложности. Сложность программного продукта объясняется рядом объективных причин. Этими причинами являются: сложность окружающего мира, трудность управления процессом разработки, гибкость программного обеспечения, так как разработчик может обеспечить себя всеми необходимыми элементами, проблемы, связанные с описанием поведения

больших дискретных систем, которыми являются прикладные программы. Рассмотрим подробнее объективные причины возникновения сложности программного продукта [1].

Сложность окружающего мира. Сложность окружающего мира объясняется «нестыковкой» между пользователем системы и ее разработчиками, пользователи, как правило, с трудом могут объяснить понятно разработчику, что необходимо сделать. У пользователя и разработчика разные взгляды на сущность проблемы способы ее решения (рис.1).

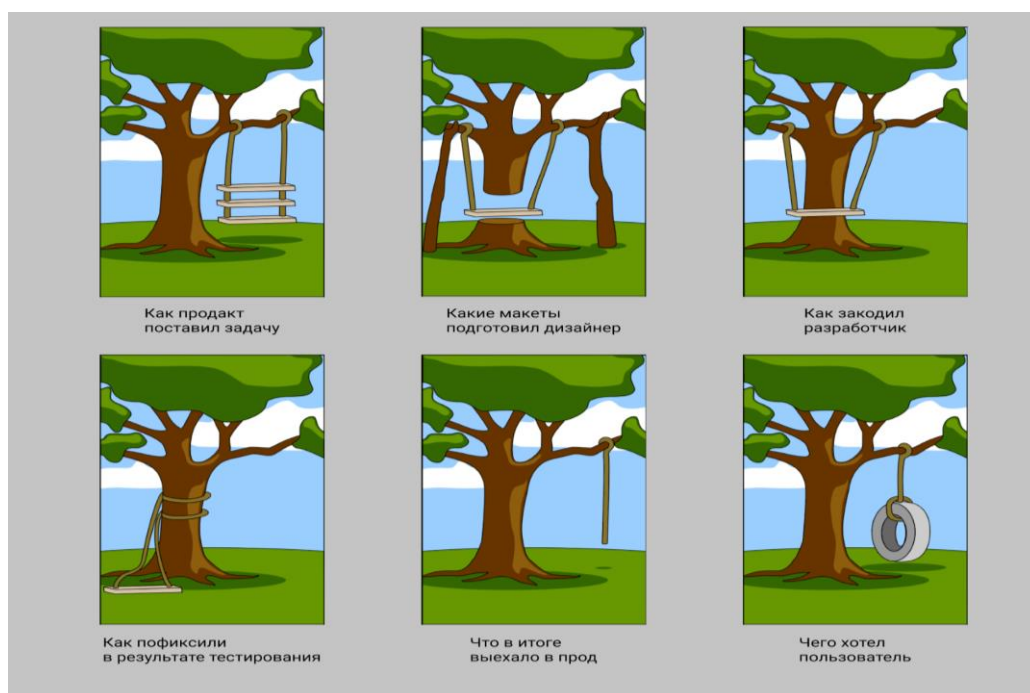


Рис.1. Проект «Качели»

Дополнительные сложности возникают в результате изменений требований к программной системе в процессе разработки. Большая программная система – это крупное капиталовложение, и поэтому необходимо сохранение на каждом шаге, большая часть затрат идет на сохранение.

Основное в формулировке задания – это вы сами должны четко представлять ожидаемый от разработчика результат.

Для формирования корректной постановки задачи необходимо собрать всю информации до того, как задача попадет к разработчику, вы должны сами полностью понять бизнес и функциональные требования.

Главное требование к задаче – прочитавший её разработчик должен сразу понять, что ему нужно делать, и это представление должно совпадать с ожиданиями менеджера. А после прочтения идеально описанной задачи у разработчика появляется четкое представление, как он будет ее выполнять.

Основная задача разработчика состоит в создании иллюзии простоты, в защите пользователей от сложности описываемого предмета или процесса. В настоящее время программные системы имеют размер, который оценивается десятками тысяч или даже миллионами строк, поэтому такой объем потребует привлечения команды разработчиков. Возникают трудности управления процессом разработки и с организацией коллективной разработки. Главная задача сохранить единство и целостность разработки.

Гибкость программного обеспечения, так как разработчик может обеспечить себя всеми необходимыми элементами в отличие от других отраслей, например, строительства, где есть блоки, из которых строится здание.

Проблемы, связанные с описанием поведения больших дискретных систем, которыми являются прикладные программы. Внутри большой прикладной программы могут существовать сотни и тысячи переменных. Исполнение программы происходит на цифровом компьютере, то имеется в каждый момент времени система с дискретными состояниями. Изменение входных параметров меняет дискретное состояние. Но состояний конечное число, которое может быть сколь угодно большим.

Каждая сложная система имеет пять признаков [5]:

1. Сложные системы часто являются иерархическими и состоят из взаимозависимых подсистем, которые в свою очередь могут быть разделены на подсистемы, и т. д., вплоть до самого низкого уровня.

2. Выбор, какие компоненты являются в данной системе элементарными, относительно произволен и оставляется на усмотрение исследователя.

3. Внутри компонентные связи обычно сильнее, чем связь между компонентами.

4. Иерархические системы обычно состоят из немногих типов подсистем, по-разному скомбинированных и организованных.

5. Любая работающая сложная система является результатом развития работающей более простой системы.

Сложность программных систем возрастает, но человеческие способности ограничены, поэтому необходима технология проектирования сложных систем, позволяющая выйти из этого затруднительного положения. При этом под проектированием будет пониматься сложный итерационный процесс, при помощи которого требования к программной системе (ПС) транслируются в инженерное представление ПС.

Одним из способов управления сложными системами является метод декомпозиции. Известны две схемы декомпозиции: алгоритмическая и объектно-ориентированная. В основе алгоритмической декомпозиции лежит разбиение по действиям-алгоритмам, объектно-ориентированная декомпозиция обеспечивает разбиение по автономным лицам – объектам реального (или виртуального) мира. Опыт показывает, что начинать необходимо с объектной декомпозиции, это поможет придать организованность сложной ПС. Объектно-ориентированное представление ПС основывается на принципах абстрагирования, инкапсуляции, модульности и иерархической организации. Совместное использование этих принципов рассчитано на проведение объектно-ориентированной декомпозиции. Объектно-ориентиро-

ванная декомпозиция снижает сложность при разработке ПС, так как она развивается из меньших систем, в работе которых уже есть уверенность. Есть два метода проектирования структурное и объектно-ориентированное, основанные на разных декомпозициях.

1.3. Модели жизненного цикла информационной системы

Жизненный цикл (ЖЦ) информационной системы характеризуется периодом времени от идеи создания информационной системы и заканчивая моментом вывода ее из эксплуатации и включает в себя следующие стадии:

1. Предпроектное обследование.
2. Проектирование.
3. Создание информационной системы.
4. Ввод в эксплуатацию.
5. Эксплуатация информационной системы.
6. Вывод из эксплуатации.

При этом каждая стадия ЖЦ сопровождается документацией. Процессы жизненного цикла, определены стандартом ГОСТ Р ИСО/МЭК 15288-2005, могут применяться любой организацией при приобретении и использовании или создании и поставке системы. Процессы, описанные в настоящем стандарте, не препятствуют и не исключают использование дополнительных процессов, которые организация посчитает необходимыми [6].

Каждая система имеет свой жизненный цикл. Жизненный цикл может быть описан с использованием абстрактной функциональной модели, представляющей концептуализацию потребности в системе, ее реализации, применения, развития и ликвидации [6].



Рис. 2. Жизненный цикл информационной системы

Жизненные циклы различаются по свойствам, целям, использованию системы, а также по преобладающим условиям. Тем не менее, несмотря на множество различий в жизненных циклах систем, существует базовый набор стадий жизненного цикла, составляющих полный жизненный цикл любой системы. Каждая стадия имеет определенную цель и вклад в полный жизненный цикл и рассматривается при планировании и выполнении жизненного цикла системы [6].

Стадии представляют собой основные периоды жизненного цикла, связанные с системой и относящиеся к состоянию описания системы или непосредственно к системе. Аналогично тому, как все системные элементы осуществляют вклад в систему как в единое целое, так и каждая стадия жизненного цикла должна учитываться на любой другой ее стадии. Следо-

вательно, участвующие стороны должны координировать свои действия и кооперироваться друг с другом на протяжении всего жизненного цикла.

Проектирование является одним из основных этапов жизненного цикла разработки информационной системы. Наибольшее распространение получили следующие модели жизненного цикла информационных систем (ИС): Waterfall - каскадная (классическая или водопадная), итерационная и спиральная. В настоящее время наиболее распространена методология управления разработкой – Agile.

Каскадная (водопадная, классическая, Waterfall) модель жизненного цикла ИС

Модель была предложена в 1970 году Уинстоном Ройсом. Данная модель предусматривает, что каждый последующий шаг начинается после полного завершения выполнения предыдущего шага (рис.3). На всех шагах модели выполняются вспомогательные и организационные процессы и работы, включающие управление проектом, оценку и управление качеством, верификацию и аттестацию, менеджмент конфигурации, разработку документации. В результате завершения шагов формируются промежуточные продукты, которые не могут изменяться на последующих шагах.

К достоинствам данной модели относятся:

- стабильность требований в течение всего жизненного цикла разработки;
- на каждой стадии формируется законченный набор проектной документации, отвечающий критериям полноты и согласованности;
- определенность и понятность шагов модели и простота её применения;
- выполняемые в логической последовательности этапы работ позволяют планировать сроки завершения всех работ и соответствующие ресурсы (денежные, материальные и людские).

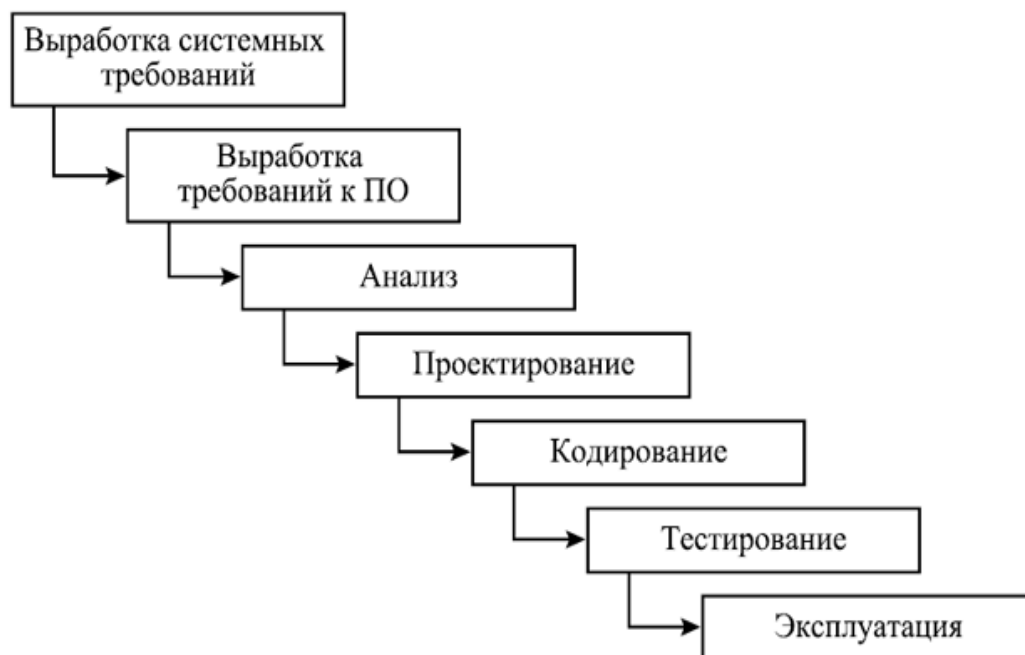


Рис.3. Каскадная модель жизненного цикла ИС

Каскадная модель хорошо зарекомендовала себя при построении относительно простых ПО, когда в самом начале разработки можно достаточно точно и полно сформулировать все требования к продукту.

Недостатки модели:

- сложность чёткого формулирования требований и невозможность их динамического изменения на протяжении пока идет полный жизненный цикл;
- низкая гибкость в управлении проектом;
- последовательность линейной структуры процесса разработки, в результате возврат к предыдущим шагам для решения возникающих проблем приводит к увеличению затрат и нарушению графика работ;
- непригодность промежуточного продукта для использования;
- невозможность гибкого моделирования уникальных систем;
- позднее обнаружение проблем, связанных со сборкой, в связи с одновременной интеграцией всех результатов в конце разработки;

- недостаточное участие пользователя в создании системы — в самом начале (при разработке требований) и в конце (во время приёмочных испытаний);

- пользователи не могут убедиться в качестве разрабатываемого продукта до окончания всего процесса разработки. Они не имеют возможности оценить качество, т.к. нельзя увидеть готовый продукт разработки;

- у пользователя нет возможности постепенно привыкнуть к системе. Процесс обучения происходит в конце жизненного цикла, когда ПО уже запущено в эксплуатацию;

- каждая фаза является предпосылкой для выполнения последующих действий, что превращает такой метод в рискованный выбор для систем, не имеющих аналогов, т.к. он не поддается гибкому моделированию.

Реализовать Каскадную модель жизненного цикла затруднительно ввиду сложности разработки ПС без возвратов к предыдущим шагам и изменения их результатов для устранения возникающих проблем. Использование данной модели наиболее эффективно в следующих случаях:

- при разработке проектов с четкими, неизменяемыми в течение жизненного цикла требованиями, понятными реализацией и техническими методиками;

- при разработке проекта, ориентированного на построение системы или продукта такого же типа, как уже разрабатывались разработчиками ранее;

- при разработке проекта, связанного с созданием и выпуском новой версии уже существующего продукта или системы;

- при разработке проекта, связанного с переносом уже существующего продукта или системы на новую платформу;

– при выполнении больших проектов, в которых задействовано несколько больших команд разработчиков.

На практике, все же приходится возвращаться к предыдущим этапам и в этом случае, в последнее время наиболее востребованной стала итерационная модель ЖЦ ИС.

Итерационная (итеративная) модель жизненного цикла ИС

Поэтапная модель с промежуточным контролем — итерационная модель разработки информационной системы. Каждый этап имеет обратные связи в процессе корректировки и создает условия для корректировки ранее созданных этапов. При этом трудоемкость работ и временные затраты существенно сокращаются по сравнению с каскадной моделью жизненного цикла (рис. 4)



Рис.4. Итерационная модель жизненного цикла ИС

В начале работы над проектом определяются все основные требования к системе, подразделяются на более и менее важные. После чего выполняется разработка системы по принципу приращений, так, чтобы разработчик мог использовать данные, полученные в ходе разработки ПО. Каждый инкремент должен добавлять системе определенную функциональность. При этом выпуск начинают с компонентов с наивысшим приоритетом. Когда части системы определены, берут первую часть и начинают её детализировать, используя для этого наиболее подходящий процесс. В то

же время можно уточнять требования и для других частей, которые в текущей совокупности требований данной работы были заморожены. Если есть необходимость, можно вернуться позже к этой части. Если часть готова, она поставляется клиенту, который может использовать её в работе. Это позволит клиенту уточнить требования для следующих компонентов. Затем занимаются разработкой следующей части системы. Ключевые этапы этого процесса — простая реализация подмножества требований к программе и совершенствование модели в серии последовательных релизов до тех пор, пока не будет реализовано ПО во всей полноте.

Достоинства и недостатки этой модели такие же, как и у каскадной (классической модели жизненного цикла). Но в отличие от классической модели заказчик может раньше увидеть результаты. Уже по результатам разработки и внедрения первой версии он может незначительно изменить требования к разработке, отказаться от нее или предложить разработку более совершенного продукта с заключением нового договора.

Достоинства:

- затраты, которые получаются в связи с изменением требований пользователей, уменьшаются, повторный анализ и совокупность документации значительно сокращаются по сравнению с каскадной моделью;
- легче получить отзывы от клиента о проделанной работе — клиенты могут озвучить свои комментарии в отношении готовых частей и могут видеть, что уже сделано. Т.к. первые части системы являются прототипом системы в целом;
- у клиента есть возможность быстро получить и освоить программное обеспечение — клиенты могут получить реальные преимущества от системы раньше, чем это было бы возможно с каскадной моделью.

Недостатки:

- менеджеры должны постоянно измерять прогресс процесса. в случае быстрой разработки не стоит создавать документы для каждого минимального изменения версии;

- структура системы имеет тенденцию к ухудшению при добавлении новых компонентов — постоянные изменения нарушают структуру системы. Чтобы избежать этого требуется дополнительное время и деньги на рефакторинг. Плохая структура делает программное обеспечение сложным и дорогостоящим для последующих изменений. А прерванный Жизненный цикл ПО приводит еще к большим потерям.

Схема не позволяет оперативно учитывать возникающие изменения и уточнения требований к ПО. Согласование результатов разработки с пользователями производится только в точках, планируемых после завершения каждого этапа работ, а общие требования к ПО зафиксированы в виде технического задания на всё время её создания.

Спиральная модель жизненного цикла ИС

Спиральная модель предложена Барри Боэм в 1988 году и определяет, в основном стартовые этапы жизненного цикла информационной системы. При этом обосновывается и проверяется возможность реализации спроектированных технических решений. На каждом витке создается прототип проектируемой информационной системы, который на следующих витках спирали ЖЦ ИС совершенствуется, дополняется и доводится до полного внедрения (рис.5).

Основным ее новым элементом является общая структура действий на каждой итерации — планирование, определение задач, ограничений и вариантов решений, оценка предложенных решений и рисков, выполнение основных работ итерации и оценка их результатов. Количество витков, а также расположение и набор видов деятельности в правом нижнем квад-

ранте могут изменяться в зависимости от результатов планирования и анализа рисков, проводимых на предыдущих этапах.

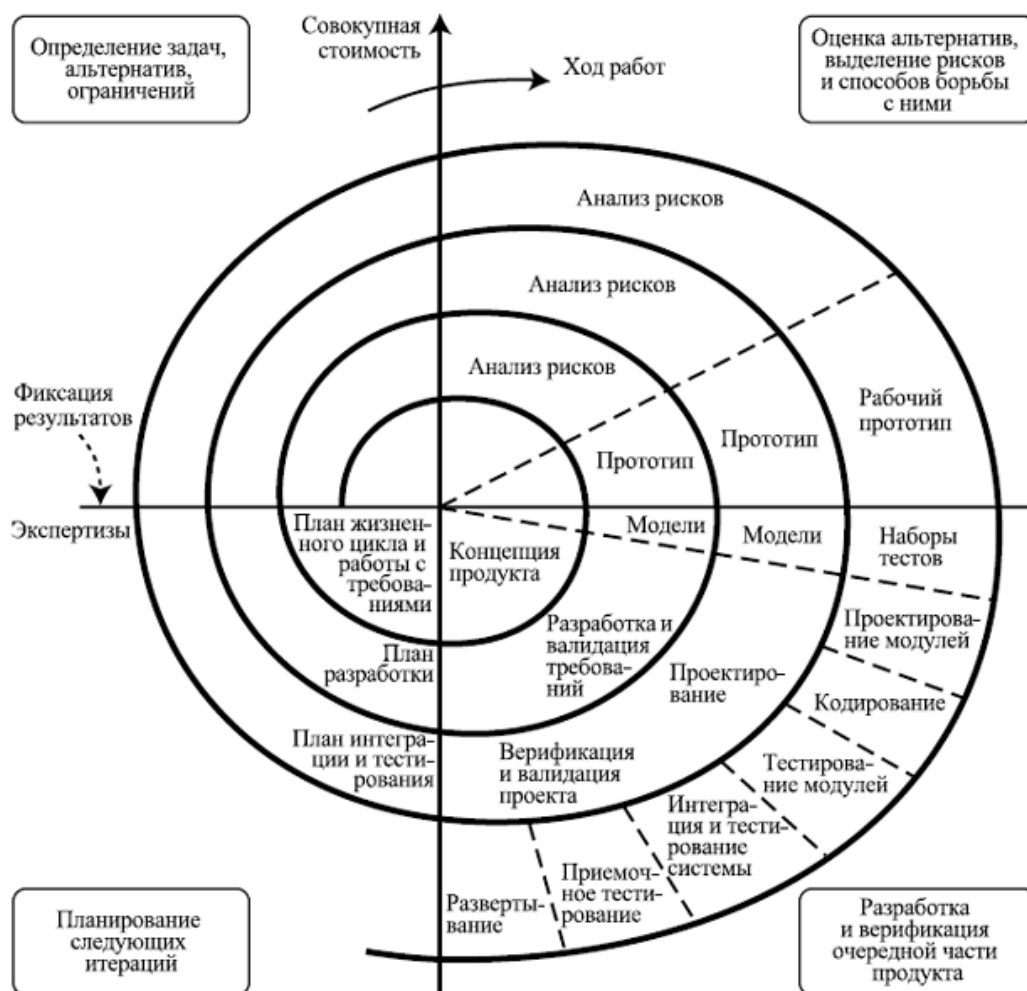


Рис.5. Спиральная модель жизненного цикла ИС

Спиральную модель часто называют мета-моделью, поскольку в ней используются два подхода: каскадная модель и модель прототипирования. Но крайне важно понимать, что спиральная модель не является простой последовательностью этапов разработки, следующих каскадной модели. На самом деле, спиральная модель является довольно гибкой. Стоит помнить, что схема, приведенная выше содержит определенные упрощения. Может показаться, что все стадии следуют одной спиральной последовательности.

Но реальный жизненный цикл ПО более гибкий, чем это изображено на схеме. Существует даже возможность вернуться к предыдущим фазам в случае необходимости пересмотра принятых решений.

Достоинства:

- мониторинг рисков является одной из главных особенностей, делающих данную модель особенно привлекательной в том случае, если вам предстоит управление большим, сложным и дорогостоящим проектом. Более того, проект будет более прозрачным, поскольку спиральная модель изначально была спроектирована таким образом, чтобы каждая итерация тщательно анализировалась;
- заказчик может увидеть работающую версию продукта уже на ранних стадиях жизненного цикла ПО;
- изменения могут быть внесены на поздних стадиях разработки;
- проект может быть разделен на несколько частей и те из них, которые, согласно анализу, окажутся более рискованными, могут быть реализованы на ранних стадиях. Такой подход может снизить трудности, связанные с управлением проектом;
- строгий контроль над документацией, как результат постоянного анализа рисков.

Недостатки:

- мониторинг рисков требует дополнительных ресурсов, а значит, эта модель может оказаться весьма затратной. Каждая итерация требует отдельной экспертизы, что делает управление проектом сложнее. Именно поэтому спиральная модель плохо подходит для небольших проектов;
- большое количество промежуточных стадий разработки. Как следствие — большой объем документации;

- на самых ранних стадиях дата завершения работы над проектом может быть неизвестна, что также усложняет контроль над процессом разработки.

Основная проблема спирального цикла — определение момента перехода на следующий этап. Для её решения вводятся временные ограничения на каждый из этапов жизненного цикла и переход осуществляется в соответствии с планом, даже если не вся запланированная работа закончена. Планирование производится на основе статистических данных, полученных в предыдущих проектах и личного опыта разработчиков. Применение спиральной модели целесообразно в следующих случаях:

- при разработке проектов, использующих новые технологии;
- при разработке новой серии продуктов или систем;
- при разработке проектов с ожидаемыми существенными изменениями или дополнениями требований;
- для выполнения долгосрочных проектов;
- при разработке проектов, требующих демонстрации качества и версий системы или продукта через короткий период времени;
- при разработке проектов для которых необходим подсчет затрат, связанных с оценкой и разрешением рисков.

Две самые популярные методологии в IT – Waterfall («водопад» / «каскадная») и Agile (в частности её ветвь - Scrum).

Agile (agile software development, от agile – проворный) — гибкая методология разработки. Возник в феврале 2001 года, когда 17 разработчиков опубликовали Манифест гибкой разработки программного обеспечения, с рассказом о «лучших способах разработки программного обеспечения». Характеризуется небольшими циклическими изменениями, которые внедряют в ответ на изменение требований. Метод появился в IT-среде, но за-

тем распространился и в другие сферы – от промышленной инженерии до искусственного интеллекта.

Особенности Agile: поощряет постоянную обратную связь от конечных пользователей вместо тщательного планирования.

Процесс: кросс-функциональные команды работают над итерациями продукта в течение определенного периода времени.

Цель: цель каждой итерации — создать рабочий продукт.

В Agile есть 4 ценности и 12 принципов.

Ценности.

1. Люди и взаимодействие важнее процессов и инструментов.
2. Работающий продукт важнее исчерпывающей документации.
3. Сотрудничество с клиентом важнее согласования условий контракта
4. Готовность к изменениям важнее следования первоначальному плану.

Для каких проектов используют Agile:

5. Когда нет уверенности в том, каким должен быть конечный результат, но есть общее представление о продукте.
6. Когда проект нужно быстро подстраивать под изменения.
7. Если взаимодействие и коммуникация — сильные стороны команды, а планирование — нет.
8. Большой и технологически сложный. Когда дешевле делать все постепенно и постоянно тестировать, чем переделывать уже готовый продукт.
9. Длительный по времени. Чем дольше проект будет функционировать, тем тяжелее представить его развитие — например, интернет-магазин.

10. С высокой неопределённостью. Когда проект инновационный, невозможно заранее продумать все функции, проще делать его маленькими рывками и тестировать.

11. Когда идей много и непонятно, какие из них окажутся удачными. Внедрять все сразу — рискованно и экономически неоправданно.

12. Когда клиент настолько заинтересован в продукте, что хочет сам во всём участвовать.

12 принципов гибкой методологии Agile.

1. Наивысшим приоритетом является удовлетворение потребностей клиента, благодаря регулярной и ранней поставке ценного программного обеспечения.

2. Изменение требований приветствуется даже на поздних стадиях разработки.

3. Работающий продукт следует выпускать как можно чаще, с периодичностью от пары недель до пары месяцев.

4. На протяжении всего проекта разработчики и представители бизнеса должны ежедневно работать вместе.

5. Над проектом должны работать мотивированные профессионалы. Чтобы работа была сделана, создайте условия, обеспечьте поддержку и полностью доверьтесь им.

6. Непосредственное общение является наиболее практичным и эффективным способом обмена информацией как с самой командой, так и внутри команды.

7. Работающий продукт — основной показатель прогресса.

8. Инвесторы, разработчики и пользователи должны иметь возможность поддерживать постоянный ритм бесконечно.

9. Постоянное внимание к техническому совершенству и качеству проектирования повышает гибкость проекта.

10. Простота — искусство, минимизации лишней работы — крайне необходима.

11. Самые лучшие требования, архитектурные и технические решения рождаются у самоорганизующихся команд.

12. Команда должна систематически анализировать возможные способы улучшения эффективности и соответственно корректировать стиль своей работы.

Scrum — это один из фреймворков Agile, «подход структуры». Над каждым проектом работает универсальная команда специалистов (+ владелец продукта и scrum-мастер). Product owner соединяет команду с заказчиком и следит за развитием проекта. Scrum-мастер помогает организовать бизнес-процесс и следит за соблюдением scrum-подхода. Scrum-подход делит рабочий процесс на равные спринты (обычно от 1 до 4 недель).

По методике Scrum формируется команда. Оптимальная команда в Scrum состоит из 7 человек. Особенность членов команды заключается в том, что в Скрам на разных этапах работы по реализации проекта они могут успешно меняться ролями и взаимозаменять друг друга. Благодаря этому возможна поддержка интереса каждого участника группы к работе.

Скрам-мастер — это один человек, который отвечает за организацию рабочего процесса; управление командой; разрешение конфликтов. Он должен помочь команде в: планировании спринтов; определении работы; отслеживании прогресса.

2 СТРУКТУРНОЕ (ФУНКЦИОНАЛЬНОЕ) ПРОЕКТИРОВАНИЕ

Методология SADT (Structured Analysis and Design Technique – методология структурного анализа и проектирования), разработанная Дугласом Т. Россом в 1969-1973 годах базируется на структурном анализе систем и графическом представлении организации в виде системы функций, которые имеют три класса структурных моделей:

1. Функциональная модель.
2. Информационная модель.
3. Динамическая модель.

Процесс моделирования по методологии SADT состоит из следующих этапов:

1. Сбор информации и анализ информации о предметной области.
2. Документирование полученной информации.
3. Моделирование (IDEF0).
4. Корректурa модели в процессе итеративного рецензирования.

Методология в настоящее время более известна как нотация IDEF0, использует формализованный процесс моделирования информационных систем и имеет следующие стадии: анализ, проектирование, реализация, объединение, тестирование, установка, функционирование.

Семейство IDEF — это семейство нотаций и методов моделирования, первоначально разработанных ВВС США как часть методологии описания рабочих процессов и информационных систем.

К семейству IDEF относятся следующие стандарты:

1. **Первое поколение методов IDEF:**
 - IDEF0 — методология функционального моделирования;
 - IDEF1 — методология моделирования информационных потоков;

- IDEF2 — методология динамического моделирования развития систем.

2. Второе поколение методов IDEF:

IDEF1X — методология построения реляционных структур (баз данных).

3. Третье поколение методов IDEF:

- IDEF3 — методология документирования процессов, происходящих в системе (**Process Description Capture Method**);

- IDEF4 — методология построения объектно-ориентированных систем (**Object-Oriented Design**);

- IDEF4/C++ — методология построения объектно-ориентированных систем (**C++ Object-Oriented Design**);

- IDEF5 — стандарт онтологического исследования сложных систем (построения словаря терминов и правил) (**Ontology Description Capture**);

Частично разработанные методы IDEF:

- IDEF6 — обоснование проектных действий (**Design Rationale Capture**);

- IDEF8 — метод разработки интерфейсов взаимодействия оператора и системы (пользовательских интерфейсов) (**Human-System Interaction Design**);

- IDEF9 — метод исследования бизнес ограничений (**Business Constraint Discovery**);

- IDEF14 — метод проектирования компьютерных сетей (**Network Design**).

Дополнительно необходимые методы IDEF (не разработаны):

- IDEF7 — аудит информационных систем (**Information System Auditing**);

- **IDEF10** — моделирование архитектуры выполнения (Information Artifact Modeling);
- **IDEF11** – метод моделирования артефактов (Information Artifact Modeling);
- **IDEF12** — организационное моделирование (Organizational Design);
- **F13** — трехсхемное проектирование преобразования данных (Three Schema Mapping Design).

2.1. Методология построения процессов IDEF0

IDEF0 (Function Modeling) – это нотация функционального моделирования процессов. Методологию IDEF0 можно считать следующим этапом развития хорошо известного графического языка описания функциональных систем SADT (Structured Analysis and Design Technique).

Проектирование информационных систем по стандарту IDEF0 сводится к декомпозиции основных функций организации на отдельные бизнес-процессы, работы или действия. В результате разрабатывается иерархическая модель анализируемой организации, при этом декомпозицию можно проводить многократно, до четкого и детального описания всех процессов. Диаграммы IDEF0 верхнего уровня принято называть родительскими, а нижнего уровня – дочерними.

В основе методологии лежат четыре основных понятия: функциональный блок, дуга (стрелка), декомпозиция, глоссарий. Функциональный блок, или работа (Activity Box) представляет собой некоторую конкретную функцию (работу) в рамках рассматриваемой системы. Блок должен иметь название в глагольном наклонении (например, "Проверить документ" или

"Проверка документа"). На диаграмме функциональный блок изображается прямоугольником (рис. 6).

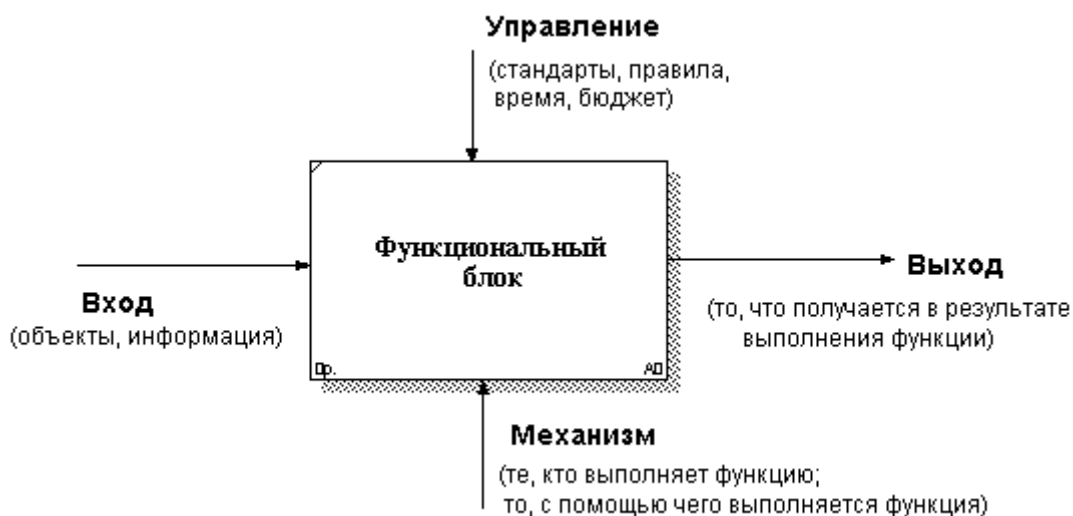


Рис.6. Функциональный блок

Каждая из четырех сторон функционального блока имеет свое определенное значение (роль) и определяет тип интерфейса, т. е. способ взаимодействия дуги с блоком: – верхняя сторона имеет значение "Управление" (Control); – левая сторона имеет значение "Вход" (Input); – правая сторона имеет значение "Выход (Output); – нижняя сторона имеет значение "Механизм" (Mechanism). Дуга (Arrow) отображает элемент системы, который обрабатывается функциональным блоком или оказывает иное влияние на функцию, представленную данным функциональным блоком. Интерфейсные дуги часто называют потоками или стрелками. С помощью дуг отображают различные объекты, которые передаются между блоками, обрабатываются блоками, определяют правила обработки и механизмы обработки. Такими объектами могут быть элементы реального мира (детали, вагоны, сотрудники и т.д.) или потоки данных и информации (документы, данные, инструкции и т.д.). Стрелки снабжаются надписями – названиями. В IDEF0 различают пять типов стрелок.

Вход (Input) – материальные объекты или информация, которые используются или преобразуются работой для получения результата (выхода). Допускается, что блок может не иметь ни одной стрелки входа.

Управление (Control) – правила, стратегии, процедуры, стандарты, ограничения на бюджет и время, которыми руководствуется работа. Каждая работа должна иметь хотя бы одну стрелку управления. Управление влияет на работу, но не преобразуется работой. В случае возникновения неопределенности в статусе стрелки (управление или вход) рекомендуется рисовать стрелку управления.

Выход (Output) – материальный объект или информация, которые производятся работой. Каждая работа должна иметь хотя бы одну стрелку выхода. Работа без результата не имеет смысла и не должна моделироваться.

Механизм (Mechanism) – ресурсы, которые выполняют работу, например персонал предприятия, станки, устройства и т. д. По усмотрению аналитика стрелки механизма могут не изображаться в модели.

Вызов (Call) – специальная стрелка, указывающая на другую модель работы. Стрелка вызова используется при расщеплении модели и указывает, что некоторая работа представлена отдельной моделью.

Таким образом, любой функциональный блок по требованиям стандарта должен иметь, по крайней мере, одну управляющую дугу и одну исходящую. То есть каждый процесс должен происходить по каким-то правилам (отображаемым управляющей дугой) и должен выдавать некоторый результат (выходящая дуга), иначе его рассмотрение не имеет никакого смысла.

Декомпозиция (Decomposition) является основным понятием стандарта IDEF0. Принцип декомпозиции применяется при разбиении сложного процесса на составляющие его функции. Декомпозиция позволяет посте-

пенно и структурировано представлять модель системы в виде иерархической структуры отдельных диаграмм, что делает ее менее перегруженной и легко усваиваемой.

Глоссарий (Glossary) – это набор соответствующих определений, ключевых слов, повествовательных изложений и т.д., которые характеризуют объект, отображенный данным элементом. Этот набор является описанием сущности данного элемента. Глоссарий дополняет наглядный графический язык, снабжая диаграммы необходимой дополнительной информацией.

Модель в методологии IDEF0 – это совокупность иерархически упорядоченных и взаимосвязанных диаграмм. Каждая диаграмма является единицей описания системы и располагается на отдельном листе. Модель может содержать четыре типа диаграмм:

1. Контекстная диаграмма, которая представляет всю систему как один блок и показывает контекст системы, т. е. связь системы с внешним миром. Модель может иметь только одну контекстную диаграмму.

2. Диаграммы декомпозиции, которые получаются в результате разбиения контекстной диаграммы на отдельные активности. Такой процесс называется функциональной декомпозицией, а диаграммы, получившиеся в результате декомпозиции, называются диаграммами декомпозиции. После декомпозиции контекстной диаграммы производится декомпозиция каждой получившейся диаграммы и т. д. Декомпозиция продолжается до достижения нужного уровня подробности описания. После каждого сеанса декомпозиции проводятся сеансы экспертизы – эксперты предметной области проверяют соответствие реальных бизнес–процессов созданным диаграммам. Найденные несоответствия исправляются, и только после прохождения экспертизы без замечаний можно приступить к следующему сеансу декомпозиции. Так достигается соответствие модели реальным бизнес–

процессам на каждом уровне модели. Синтаксис описания системы в целом и каждого ее фрагмента одинаков во всей модели.

3. Диаграммы дерева узлов показывают иерархическую зависимость работ. То есть, в виде дерева показывается, какие активности получились в результате декомпозиции каждой активности. Диаграмм деревьев узлов может быть в модели несколько, поскольку дерево может быть построено на различную глубину и начиная с любой диаграммы (не обязательно с контекстной).

4. Диаграммы только для экспозиции (FEO – for exposition only) строятся для иллюстрации альтернативной точки зрения, для хранения старых версий. FEO – это просто картинка. Дело в том, что методология не поддерживает альтернативные варианты декомпозиции. Если необходимо хоть как-то сохранить альтернативный вариант декомпозиции, то применяют диаграмму только для экспозиции.

Модель IDEF0 всегда начинается с представления системы как единого целого – одной активности с дугами, простирающимися за пределы рассматриваемой области. Такая диаграмма с одной активностью называется контекстной диаграммой. Дуги контекстной диаграммы должны описывать все основные связи моделируемой системы с внешним миром. Она также может называться родительской диаграммой [7].

Перед началом процесса проектирования ИС должна быть поставлена цель моделирования. Цель должна отвечать на следующие вопросы:

1. Почему этот процесс должен быть смоделирован?
2. Что должна показывать модель?
3. Что может получить читатель?

Точка зрения. Несмотря на то, что при построении модели учитываются мнения различных людей, модель должна строиться с единой точки зрения. Точку зрения можно представить как взгляд человека, который ви-

дит систему в нужном для моделирования аспекте. Точка зрения должна соответствовать цели моделирования. В течении моделирования важно оставаться на выбранной точке зрения.

В данном примере (рис.7) субъектом будет выступать само предприятие, а именно процессы, происходящие внутри него; цель моделирования – воспроизвести бизнес-процессы, происходящие на предприятии (модель AS-IS); точка зрения – с позиции директора как лица, знающего структуру предприятия в целом.

После определения контекста моделирования можно приступить к построению контекстной диаграммы (называемой еще "черным ящиком"). Данный тип диаграммы позволяет показать, что подается на вход работы и что является результатом работы, без детализации ее составляющих. Данная диаграмма содержит только одну работу, которая будет представлять всю деятельность предприятия в целом. На рисунке 7 приведена контекстная диаграмма деятельности предприятия по сборке и продаже компьютеров и ноутбуков [7].

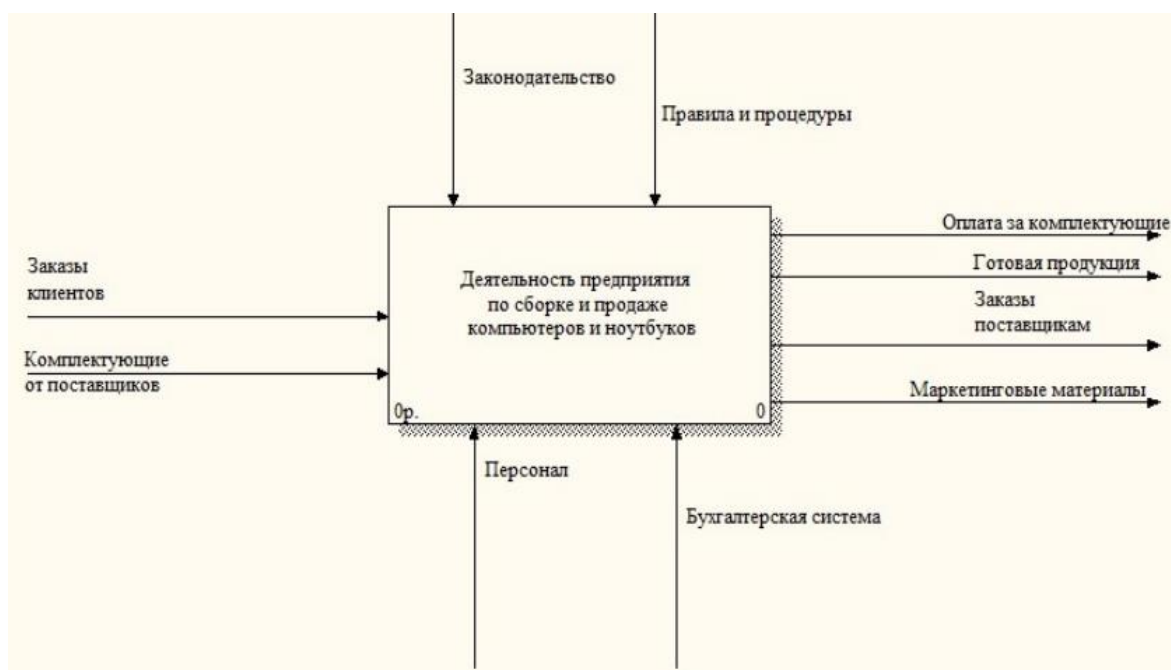


Рис.7. Контекстная диаграмма IDF0 деятельности предприятия по сборке и продаже компьютеров и ноутбуков

Входными стрелками на диаграмме являются:

- заказы клиентов - список компьютеров и их конфигурация, которые клиент желает приобрести;
- комплектующие от поставщиков - комплектующие, полученные от поставщиков, из которых собираются компьютеры и ноутбуки.

Выходными стрелками являются:

- готовая продукция - собранные компьютеры и ноутбуки;
- заказы поставщикам - список комплектующих, которые предприятие закупает у поставщиков;
- оплата за комплектующие - деньги поставщикам за комплектующие;
- маркетинговые материалы - прайс-листы, рекламки и т.п.

Стеки управления:

- законодательство - различные законодательные документы, которыми руководствуется предприятие в процессе своей деятельности;
- правила и процедуры - различные правила и процедуры, которыми руководствуется предприятие в процессе своей деятельности (например, правила сборки и тестирования компьютеров, процедура общения с клиентами и т.п.).

Стрелки механизмов:

- бухгалтерская система;
- персонал.

С помощью диаграммы декомпозиции первого уровня покажем, из каких более мелких работ состоит работа "Деятельность предприятия по сборке и продаже компьютеров и ноутбуков". В данной работе были выделены следующие дочерние работы (рис.8).

1. «Управление» - данная работа включает в себя общее управление предприятием, финансами, кадрами, бухгалтерию и т.п..
2. «Продажи и маркетинг» - работа с клиентами, презентации, выставки, реклама, маркетинговые исследования и т.д.
3. «Сборка и тестирование компьютеров» - сборка и тестирование настольных компьютеров и ноутбуков.
4. «Отгрузка и снабжение» - снабжение предприятия необходимыми комплектующими, хранение и отгрузка готовой продукции.

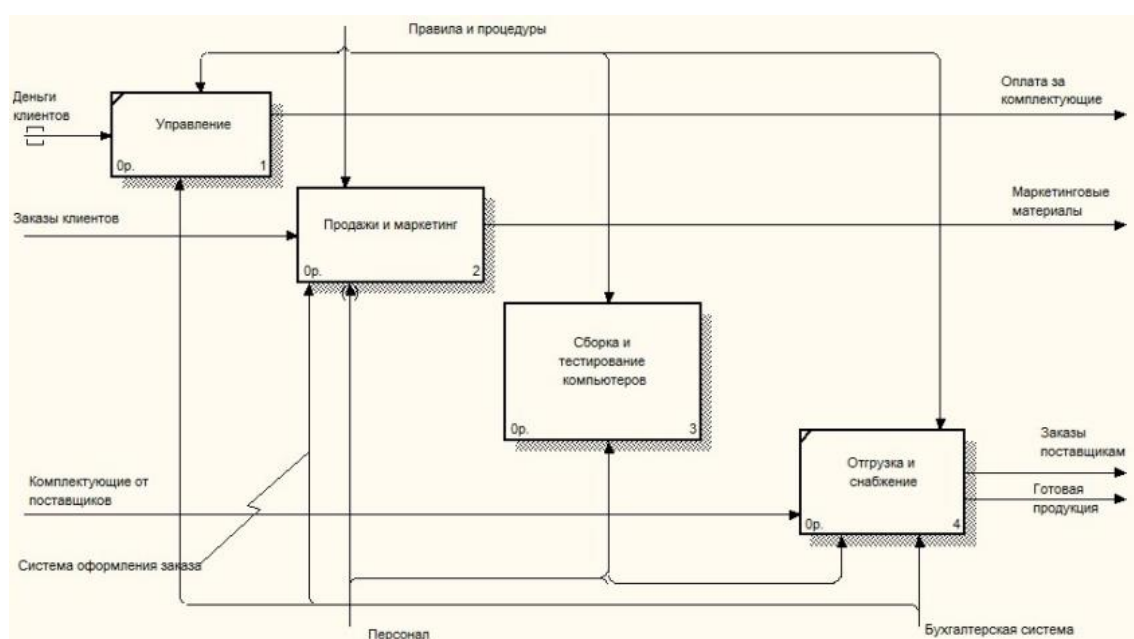


Рис.8. Декомпозиция IDF0 первого уровня, только с граничными стрелками.

После создания дочерней диаграммы первым действием соединим граничные стрелки с работами. Стрелку «Заказы клиентов» соединим с работой «Продажи и маркетинг», стрелку «Комплектующие от поставщиков» - с «Отгрузка и снабжение». Выходом работы «Управление» будет «Оплата за комплектующие», выходом «Продажи и маркетинг» «Маркетинговые материалы». Стрелки «Заказы поставщикам» и «Готовая продукция» - выход работы «Отгрузка и снабжение».

Стрелка *«Персонал»* будет являться входом механизма всех четырех работ, а стрелка *«Бухгалтерская система»* - работ *«Продажи и маркетинг»* и *«Отгрузка и получение»*. Стрелка *«Правила и процедуры»* будет входом управления всех четырех работ.

Любую ветвь стрелки также можно декомпозировать и дать ей свое название. Покажем это на примере ветки стрелки *«Бухгалтерская система»* для работы *«Продажи и маркетинг»*. Назовем ее *«Система оформления заказа»*

На данном этапе построения диаграммы выяснилось, что мы не учли такой важный фактор, как деньги, которые клиенты дают за готовую продукцию. Деньги клиентов - это вход работы *«Деятельность предприятия по сборке и продаже компьютеров и ноутбуков»*. Добавим эту стрелку на диаграмму декомпозиции и .

Если по каким-то причинам граничную стрелку дочерней диаграммы не следует показывать (например, на данной диаграмме она является несущественной, или чтоб не загромождать диаграмму), то ее можно просто удалить. На данной диаграмме мы удалили стрелку *«Законодательство»*. Стрелки которые вновь добавляли на диаграмму отображаются в квадратных скобках – их необходимо или перенести на родительскую диаграмму или затоннелировать. Если их отсутствие на родительской диаграмме не критично или они нужны только при декомпозиции, то такие стрелки тоннелируют (стрелка на диаграмме в круглых скобках).

После соединения граничных стрелок с работами следующим шагом соединим работы между собой. Поскольку работа *«Управление»* включает в себя общее управление предприятием, то одним из ее результатов будет являться *«Управляющая информация»*, поступающая на вход управления всех остальных работ.

Работа *«Продажи и маркетинг»* получает на входе заказы клиентов (т.е. количество компьютеров и их конфигурация), информацию о которых она передает работе *«Сборка и тестирование компьютеров»* в качестве управляющей информации.

Работе *«Сборка и тестирование»* для своего функционирования необходимы комплектующие, которые она заказывает у работы *«Отгрузка и снабжение»* (выходная стрелка *«Список необходимых комплектующих»*). Собранные компьютеры она также передает работе *«Отгрузка и снабжение»* (выходная стрелка *«Собранные компьютеры»*). Информация о результатах сборки и тестирования необходима работе *«Продажи и маркетинг»* (выходная стрелка *«Результаты сборки и тестирования»*).

Результатом работы *«Отгрузка и снабжение»* будут необходимые комплектующие, которые поступают на вход работы *«Сборка и тестирование компьютеров»*.

Управление любого предприятия должно знать, что происходит на предприятии, чем занимается каждое подразделение и каковы результаты их работы, т.е. любая работа в идеале должна отчитываться о результатах своей деятельности перед управлением. Создадим стрелки выходов работ *«Продажи и маркетинг»*, *«Сборка и тестирование компьютеров»* и *«Отгрузка и снабжение»* и соединим их со входом управления работы *«Управление»*.

Теперь получили полную диаграмму декомпозиции, представленную на рисунке 9.

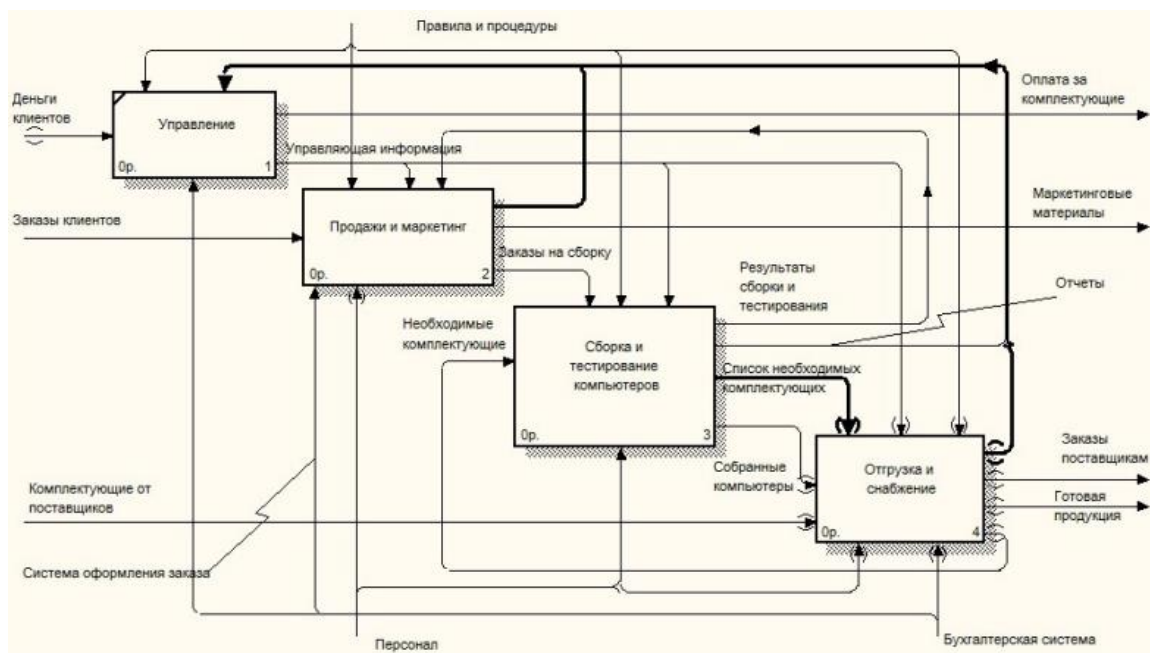


Рис.9. Декомпозиция IDF0 первого уровня

Также можем совершить и декомпозицию отдельной деятельности (работы), декомпозицию второго уровня. Проведем декомпозицию работы «Сборка и тестирование компьютеров» диаграммы А0.

В результате проведения экспертизы получена следующая информация:

1. Производственный отдел получает заказы клиентов от отдела продаж по мере их поступления.
2. Диспетчер координирует работу сборщиков, сортирует заказы, группирует их и дает указание на отгрузку компьютеров, когда они готовы.
3. Каждые 2 часа диспетчер группирует заказы - отдельно для настольных компьютеров и ноутбуков - и направляет на участок сборки.
4. Сотрудники участка сборки собирают компьютеры согласно спецификациям заказа и инструкциям по сборке. Когда группа компьютеров, соответствующая группе заказов, собрана, она направляется на тестирование.
5. Тестировщики тестируют каждый компьютер и в случае необходимости заменяют неисправные компоненты. Тестировщики направляют

результаты тестирования диспетчеру, который на основании этой информации принимает решение о передаче компьютеров, соответствующих группе заказов, на отгрузку.

Исходя из имеющихся данных в данной работе выделим четыре дочерние работы:

- «Отслеживание расписания и управление сборкой и тестированием»;
- «Сборка настольных компьютеров»;
- «Сборка ноутбуков»;
- «Тестирование компьютеров».

Также начнем с соединения граничных стрелок с работами (рис.10).

1. Стрелка *«Необходимые комплектующие»* - это вход работ *«Сборка настольных компьютеров»* и *«Сборка ноутбуков»*.

2. Стрелки управления *«Управляющая информация»* и *«Заказы на сборку»* соединим с работой *«Отслеживание расписания и управление сборкой и тестированием»*, поскольку именно данная работа управляет всем процессом сборки и тестирования, а стрелку управления *«Правила и процедуры»* - с остальными тремя работами.

3. Персонал принимает участие во всех выделенных дочерних работах, поэтому заводим стрелку *«Персонал»* на вход механизма всех работ (при этом указав, что в первой работе участвует диспетчер, а в четвертой - тестировщик).

4. Список необходимых комплектующих - это один из результатов работ *«Сборка настольных компьютеров»* и *«Сборка ноутбуков»*. Результаты сборки и тестирования - это выходы работ *«Сборка настольных компьютеров»*, *«Сборка ноутбуков»* и *«Тестирование компьютеров»*.

5. Компьютеры считаются собранными после того, как они успешно прошли тестирование, поэтому стрелка выхода *«Собранные ком-*

пьютеры» - выход работы «Тестирование компьютеров». Различные отчеты формирует работа «Отслеживание расписания и управление сборкой и тестированием».

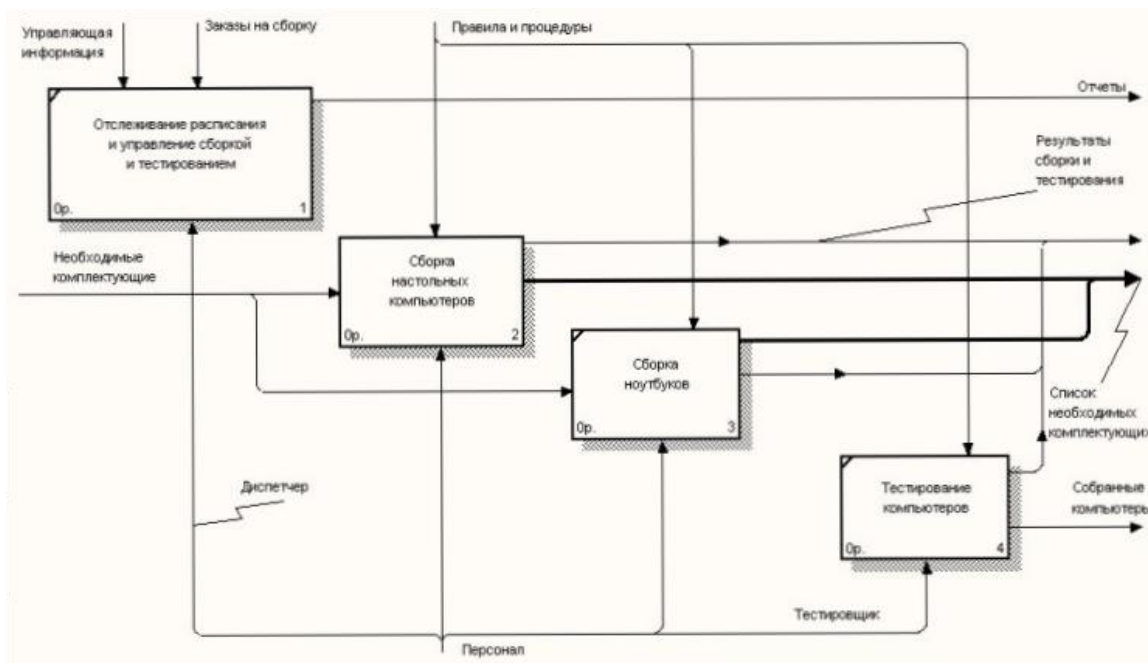


Рис.10. Декомпозиция деятельности «Сборка и тестирование компьютеров», только с граничными стрелками.

После соединения граничных стрелок с работами следующим шагом соединим работы между собой (рис 11). Поступающие заказы на сборку сортируются диспетчером, после чего они поступают на вход управления работ «Сборка настольных компьютеров» и «Сборка ноутбуков» (стрелки «Заказы на настольные компьютеры» и «Заказы на ноутбуки», соответственно). Когда компьютеры собраны, диспетчер дает указание на их отгрузку (стрелка «Указание передать компьютеры на отгрузку»).

Собранные компьютеры (выходы работ «Сборка настольных компьютеров» и «Сборка ноутбуков») должны быть протестированы, для чего они должны поступать на вход работы «Тестирование компьютеров» - стрелки «Настольные компьютеры» и «Ноутбуки».

The diagram illustrates the organizational structure of a computer assembly and testing workshop, showing four main functional blocks and their interactions with external inputs and outputs.

- Block 1: Отслеживание расписания и управление сборкой и тестированием (Order tracking and management of assembly and testing)**
 - Inputs: Управляющая информация (Controlling information), Заказы на сборку (Assembly orders).
 - Outputs: Правила и процедуры (Rules and procedures), Заказы на настольные компьютеры (Desktop computer orders), Заказы на ноутбуки (Laptop orders), Указание передать компьютеры на отгрузку (Instruction to transfer computers for shipment), Отчеты (Reports).
- Block 2: Сборка настольных компьютеров (Desktop computer assembly)**
 - Inputs: Необходимые комплектующие (Required components), Заказы на настольные компьютеры (Desktop computer orders).
 - Outputs: Результаты сборки и тестирования (Assembly and testing results), Список необходимых комплектующих (List of required components).
- Block 3: Сборка ноутбуков (Laptop assembly)**
 - Inputs: Заказы на ноутбуки (Laptop orders), Необходимые комплектующие (Required components).
 - Outputs: Результаты сборки и тестирования (Assembly and testing results), Список необходимых комплектующих (List of required components).
- Block 4: Тестирование компьютеров (Computer testing)**
 - Inputs: Настольные компьютеры (Desktop computers), Ноутбуки (Laptops), Результаты сборки и тестирования (Assembly and testing results), Список необходимых комплектующих (List of required components).
 - Outputs: Собранные компьютеры (Assembled computers), Результаты тестирования (Testing results).

External Interactions:

- Диспетчер (Dispatcher):** Provides necessary components to Blocks 2 and 3, and receives testing results from Block 4.
- Персонал (Personnel):** Provides rules and procedures to Block 1, and receives assembled computers from Block 4.
- Тестировщик (Tester):** Provides testing results to Block 4.

ПРИМЕРЫ

40

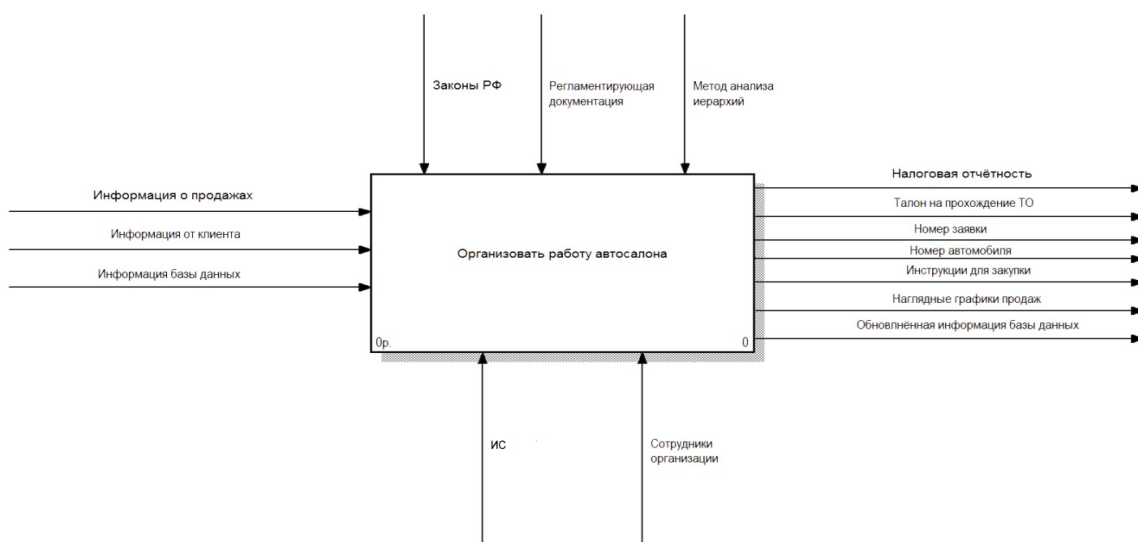


Рис.12. Контекстная диаграмма IDEF0 диаграмма верхнего уровня функциональной модели предприятия «Автосалон»

Первый уровень декомпозиции представлен пятью основными функциями предприятия: организация работы бухгалтерии, система оформления заказов, работа службы технического осмотра (ТО), система записи на тест-драйв, система расчёта статистики.

Одна из важнейших функций – это оформление новых заказов. Данная функция разбивается на 4 подфункции (рис.16). На основе переданных клиентом персональных данных информационная система производит поиск в своей базе данных для проверки существования записей о данном клиенте. Если записей не находится, то ИС добавляет в базу данные нового клиента. Добавив информацию, система получает уникальный идентификатор клиента и передаёт его функции выбора автомобиля на основе СППР. После выбора автомобиля система получает уникальный идентификатор выбранного автомобиля и вместе с идентификатором клиента эта информация передаётся в функцию оформления заказа, выходом которой является заполненный договор.

Функция выбора автомобиля с помощью СППР в свою очередь разбивается на 5 подфункций (рис.13). На основе предпочтений клиента формиру-

ется иерархия выбора автомобиля. По данной иерархии и на основе правил МАИ формируется матрица попарных сравнений и рассчитываются локальные векторы приоритетов. С помощью формулы расчёта глобальных приоритетов и рассчитанных на предыдущем этапе локальных векторов приоритетов производится выборка ограниченного числа подходящих автомобилей. После этого клиент с помощью менеджера производит окончательный выбор автомобиля, из предложенного ему списка, данные автомобиля ассоциируются с клиентом для дальнейшего учёта (рис.14-16).

Функция оформления заказа разбивается на несколько этапов при условии выбора автомобиля (рис.14- 16).

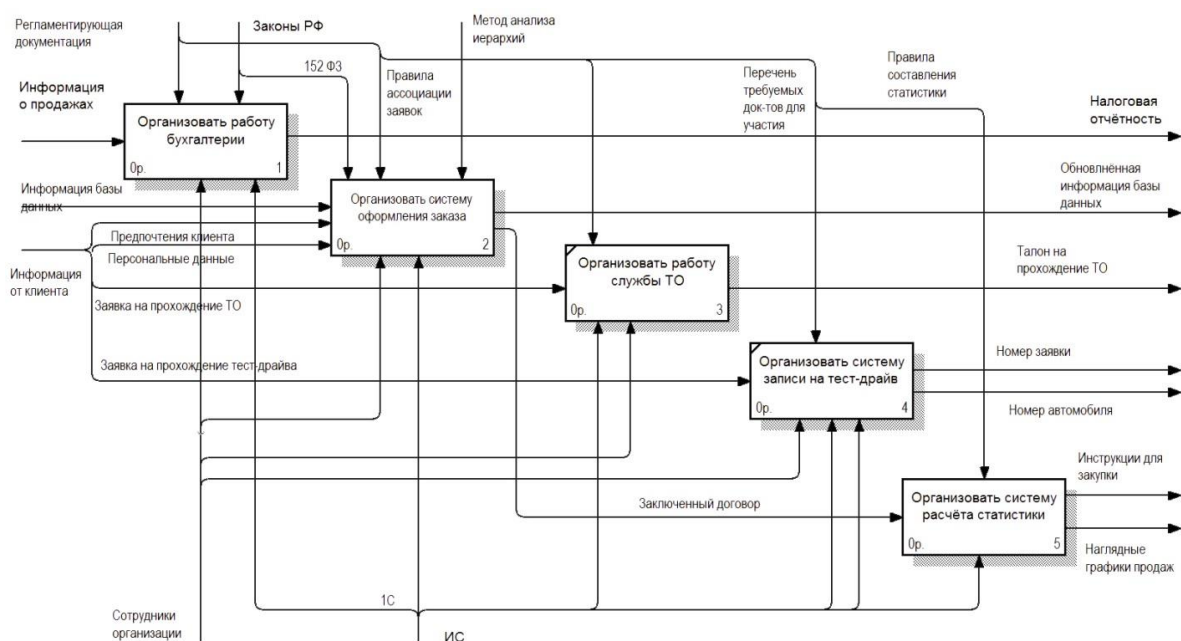


Рис. 13. Декомпозиция диаграммы верхнего уровня функциональной модели предприятия

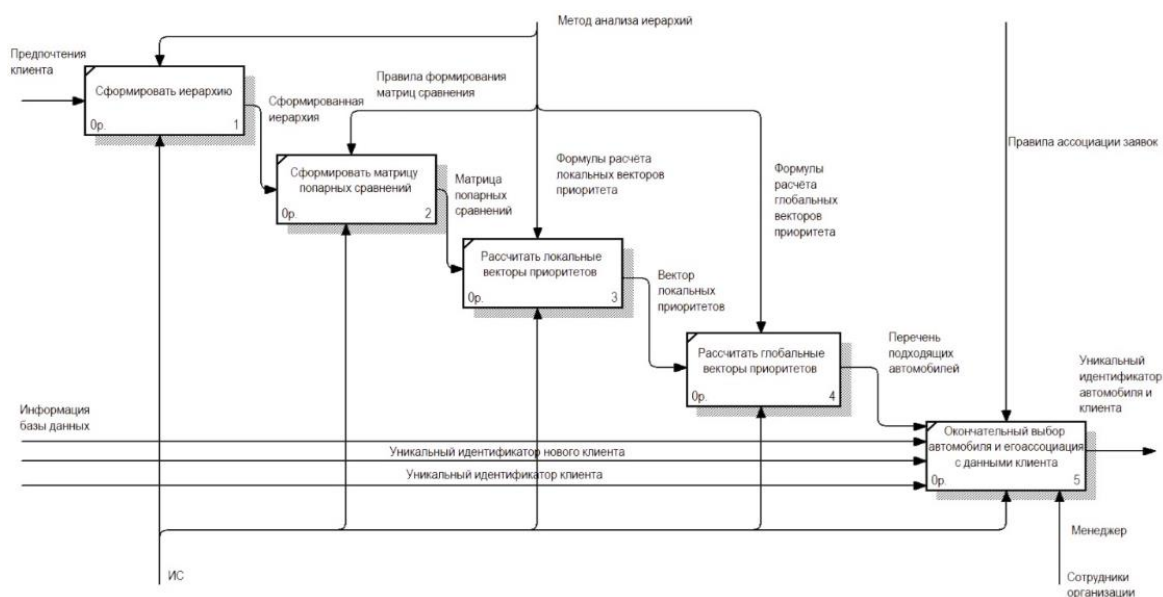


Рис.14. Декомпозиция функции организации «Оформление заказа»

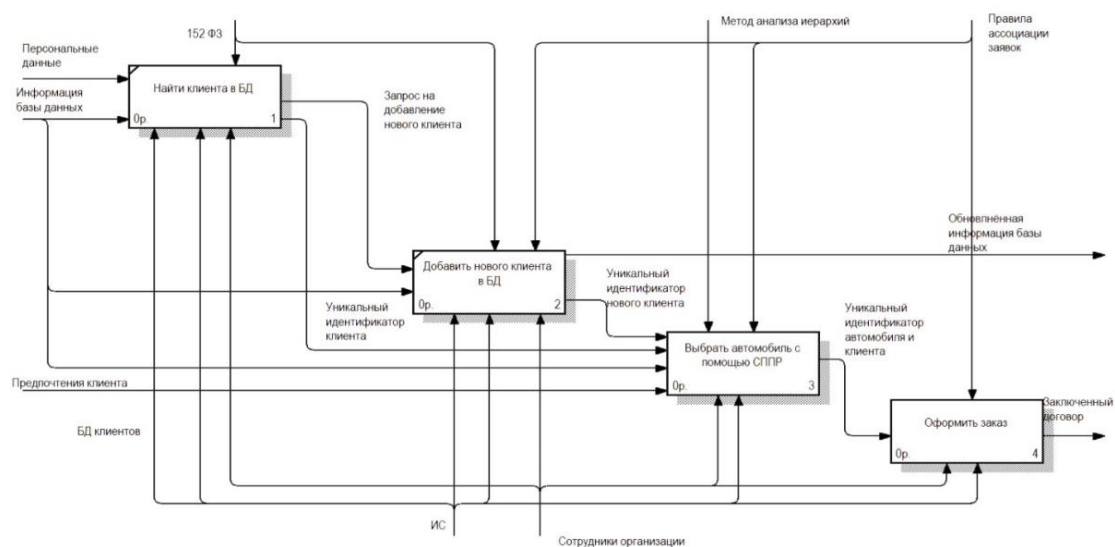


Рис.15. Декомпозиция функции «Выбор автомобиля» с помощью системы поддержки принятия решения

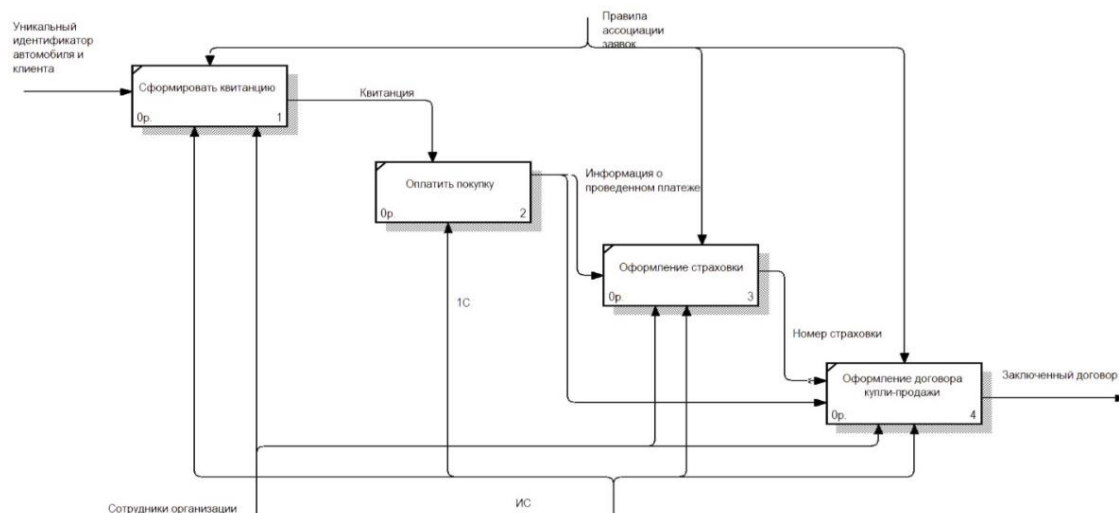


Рисунок 16. Декомпозиция функции «Оформления заказа»

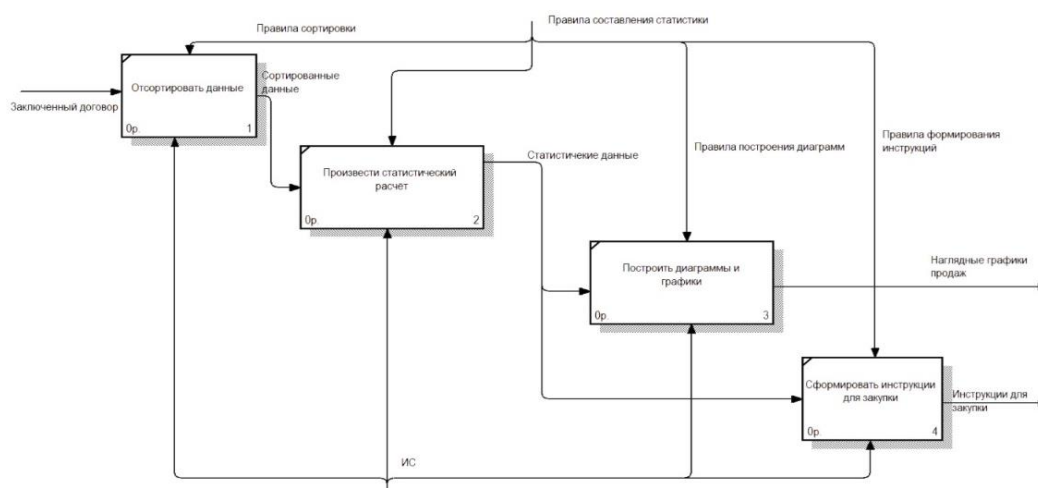


Рис.17. Декомпозиция функции «Организовать систему расчета статистики»

Получив уникальный идентификатор клиента и автомобиля, ИС формирует квитанцию для оплаты. В случае если клиент желает оформить страховку, система подготавливает необходимые документы, в противном случае оформляется договор купли-продажи.

Система расчёта статистики разбивается на 4 подфункции (рис.17), получив информацию по заключенным договорам, ИС сортирует данные и на основе правил составляет статистический расчёт. На основе полученного

расчёта строятся диаграммы графики, а также формируются инструкции для закупки определённых моделей автомобилей.

Пример 2. Информационная система «Интернет аптека»

ИС интернет-аптека обладает следующими возможностями:

- хранение данных о лекарствах, заказах, клиентах;
- корректировка данных о лекарствах, заказах, клиентах;
- поиск лекарства по наименованию, производителю, характеристике, наличию в аптеках, цене, правилам отпуска лекарства;
- добавление новых лекарств, заказов, клиентов;
- поиск справочных данных по лекарствам;
- регистрация новых клиентов;
- формирование статистической информации за отчётный период.

Входные данные системы:

- данные о лекарствах (наименование, фотография упаковки лекарства, производитель, характеристики, наличие в аптеках, цена, правила отпуска лекарства);
- личные данные клиента (фамилия, имя, дата рождения, номер телефона, электронная почта);
- данные об аптеках (адрес, график работы, фотография).

Выходные данные системы:

- статистическая информация о лекарствах (статистика просмотров, заказов);
- статистическая информация о клиентах (история просмотра лекарств, история заказов лекарств);
- справочная информация:

- данные о лекарствах (наименование, фотография упаковки лекарства, производитель, характеристики, наличие в аптеках, цена, правила отпуска лекарства);

- данные об аптеках (адрес, график работы, фотография).

Контекстная диаграмма A0 (рис.14).

Работа: Деятельность аптеки.

Входные стрелки:

- *«Заказы от покупателей»;*
- *«Лекарства от поставщиков».*

Выходные стрелки:

- *«Готовые заказы для покупателей»;*
- *«Маркетинговые материалы, заказы»;*
- *«Заказы поставщикам»;*
- *«Оплата лекарств поставщикам»;*

Стрелки управление:

- *«Законодательство»;*
- *«Правила и процедуры».*

Стрелки механизмы:

- *«Бухгалтерская система»;*
- *«Персонал».*

Работа «Деятельность аптеки» декомпозируется на 5 деятельностей (работ)

1. Управление: финансовая деятельность, организация процессов

Входные данные:

- *«Деньги покупателей».*

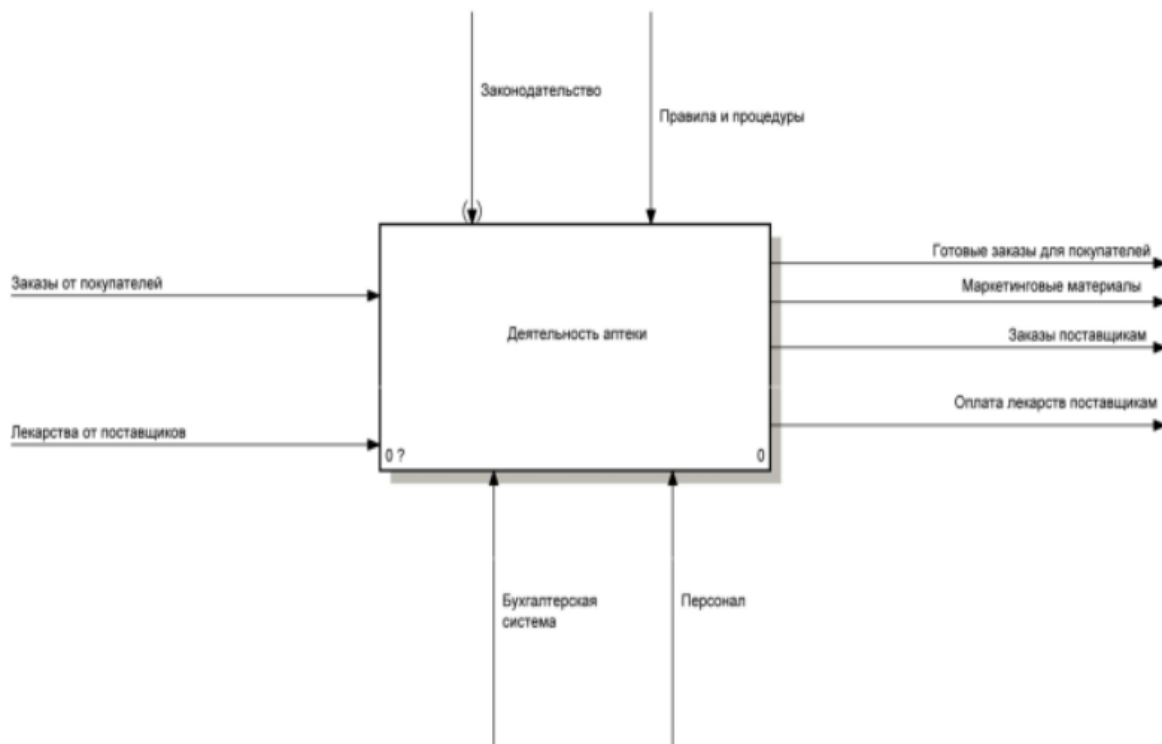


Рис.18. Контекстная диаграмма IDEF0 функциональной модели предприятия «Интернет- аптека».

Выходные данные:

- «Оплата лекарств поставщикам»;
- «Управляющая информация».

Управления:

- «Правила и процедуры».
- Механизмы:
- «Бухгалтерская система».

2. Продажи и маркетинг: связь с клиентами, связующее звено между клиентами и аптекой, отдел маркетинга

Входные данные:

- «Заказы от покупателей»;

Выходные данные:

- «Заказы на сборку»;
- «Маркетинговые материалы».

Управление:

- *«Правила и процедуры»;*
- *«Управляющая информация»;*
- *«Результат сборки».*

Механизмы:

- *«Бухгалтерская система»;*
- *«Персонал».*

3. Сборка и транспортировка заказа

Входные данные:

- *«Поставленные лекарства».*

Выходные данные:

- *«Список лекарств для поставки»;*
- *«Результат сборки»;*
- *«Заказ готовый к выдаче».*

Управление:

- *«Заказ на сборку»;*
- *«Управляющая информация»;*
- *«Правила и процедуры».*

Механизмы:

- *«Бухгалтерская система»;*
- *«Персонал».*

4. Хранение и выдача заказов(аптека): непосредственно конкретные аптеки, в которые приходит клиент

Входные данные:

- *«Заказ готовый к выдаче».*

Выходные данные:

- *«Готовые заказы для покупателей (т.е. выполненные заказы, которые забрал клиент)».*

Управление:

- *«Правила и процедуры»;*
- *«Управляющая информация».*

Механизмы:

- *«Бухгалтерская система»;*
- *«Персонал».*

5. Поставка лекарств: поставка лекарств в конкретную аптеку (подробнее о деятельности в след. лабораторной)

Входные данные:

- *«Лекарства от поставщиков».*

Выходные данные:

- *«Заказы поставщикам»;*
- *«Поставленные лекарства».*

Управление:

- *«Правила и процедуры»;*
- *«Управляющая информация»;*
- *«Список лекарств для поставки».*

Механизмы:

- *«Бухгалтерская система»;*
- *«Персонал».*

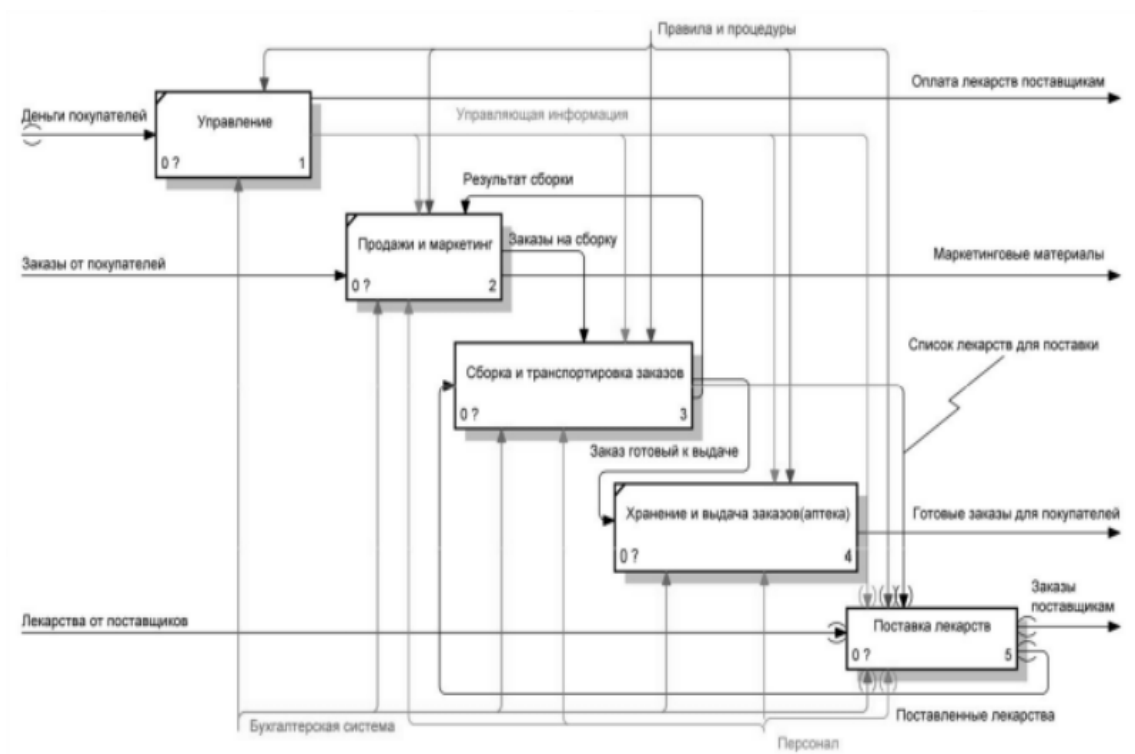


Рис.19. Декомпозиция первого уровня функциональной модели предприятия «Интернет аптека»

Декомпозиция деятельности: Сборка и транспортировка заказов

Декомпозиция происходит на 3 деятельности (рис. 16).

1. Управление сборкой

Входные данные:

- «Уведомления о начале и окончании сборки, ошибках на этапе сборки (например, отсутствие лекарства на складе)»;
- «Уведомления о ходе транспортировки (собрано, в пути, доставлено)».

Выходные данные:

- «Отчёты»;
- «Результат сборки»;
- «Указание передать заказ на выдачу»;
- «Список заказов на сборку».

Управление:

- *«Заказы на сборку»;*
- *«Управляющая информация».*

Механизмы:

- *«Бухгалтерская система».*

2. Сборка заказов:

- *получение заказа от клиента;*
- *уточнение наличия лекарств на складе выбранной клиентом аптеки;*
- *в случае отсутствия лекарств в выбранной аптеке формулируется список лекарств для поставки;*
- *в зависимости от успешности поставки собирается заказ или отправляется уведомление о невозможности поставки.*

Входные данные:

- *«Список заказов на сборку»;*
- *«Поставленные лекарства»;*
- *«Уведомления о невозможности поставки».*

Выходные данные:

- *«Список лекарства для поставки»;*
- *«Уведомления о начале и окончании сборки, ошибках на этапе сборки (например, отсутствие лекарства на складе)»;*
- *«Собранный заказ».*

Управление:

- *«Правила и процедуры».*

Механизмы:

- *«Бухгалтерская система»;*
- *«Персонал».*

3. Транспортировка собранных заказов: транспортировка заказов в конкретную аптеку

Входные данные:

- «Собранный заказ».

Выходные данные:

- «Заказ готовый к выдаче»;
- «Уведомления о ходе транспортировки (собрано, в пути, доставлено)».

Управление:

- «Правила и процедуры»;
- «Указание передать заказ на выдачу».

Механизмы:

- «Бухгалтерская система»;
- «Персонал».

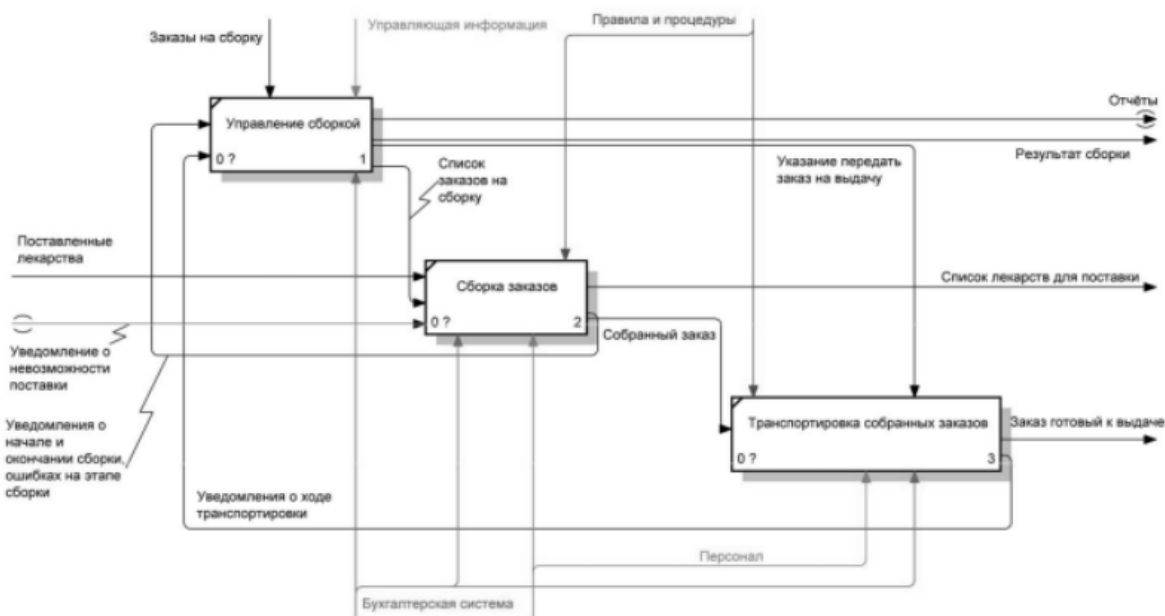


Рис.20. Декомпозиция деятельности: «Сборка и транспортировка заказов»

2.2 Методология IDEF 3

Стандарт IDEF3 базируются на концепции моделирования бизнес-процессов организации.

Бизнес-процесс – это совокупность технологических и организационно-деловых процессов, выполняемая целенаправленно в рамках заранее заданной организационной структуры. При этом важнейшим атрибутом процесса является время. Иными словами, в отличие от IDEF0, IDEF3 предназначен для моделирования деятельности, развернутой во времени.

Стандарт IDEF3 – это методология описания процессов, рассматривающая последовательность выполнения и причинно-следственные связи между ситуациями и событиями для структурного представления знаний о системе. При помощи IDEF3 описывают логику выполнения работ, очередность их запуска и завершения, т.е. IDEF3 предоставляет инструмент моделирования сценариев действий сотрудников организации, отделов, цехов и т.п., например, порядок обработки заказа или события, на которые необходимо реагировать за конечное время, выполнение действий по производству товара и т.д. Таким образом, IDEF3 – это методология, способная фиксировать и структурировать описание работы системы. При этом сбор сведений может производиться из многих источников, что позволяет зафиксировать информацию от экспертов о поведении системы. Описание процесса представляет всевозможные ситуации (процессы, функции, действия, акты, события, сценарии, процедуры, операции или решения), которые могут происходить в моделируемой системе в логических и временных отношениях. Каждый процесс представлен полем, отображающим название процесса. Номер идентификатора процесса назначается последовательно. Элемент связи необходим для организации отношений между элементами диаграммы и описания динамики происходящих процессов. Связи использу-

ются прежде всего для обозначения отношений между функциональными элементами, отображения временной последовательности выполнения сценариев в диаграммах описания процесса. Данные элементы используются в основном для обозначения существенных связей. Связи между функциональными блоками могут быть: временные, логические, причинно-следственные, природные и обычные. В подавляющем большинстве случаев используются связи, отражающие простое временное отношение между блоками. Существует два основных типа связей, используемых в IDEF3 схемах: связи приоритета (старшинства) и относительные (прерывистые) связи. Связи простой очередности демонстрируют временной приоритет отношений между функциональными блоками. Они являются наиболее широко используемыми связями и обозначаются сплошной стрелкой, а иногда дополнительным маркером, прикрепленным к стволу стрелки.

На диаграмме IDEF3 используется 5 элементов:

- *блоки действия или работы;*
- *стрелки или связи (три типа стрелок);*
- *перекрёстки;*
- *объекты ссылок.*

Рассмотрим объекты более подробно:

- **Работа, действие (Activity).** Отображает функции единицы работ (UOW unit of work), действие, работа или событие процесса (рис.21).



Рис. 21. Изображение и нумерация дейтсвия (работы) в даграмме IDEF3.
Стрелки (Arrows). Отображают взаимоотношения работ. Существует три типа стрелок, представленных в таблице 1

Таблица 1. Типы связей в модели IDEF3

Изображение	Название связи	Назначение
	Временное предшествование (Temporal precedence)	Показывает, что работа-источник должна закончиться прежде, чем работа- цель начнется
	Объектный поток (Object flow)	Применяется для описания того факта, что объект используется в двух или более единицах работы, например, когда объект порождается в одной работе и используется в другой.
	Нечеткое отношение (Relationship)	Используемая для изображения связей между единицами работ, исходным и конечным действием

Связь потоков объектов. Стрелка с двумя наконечниками, **Перекрестки.**

Перекрестки используются для отображения логики отношений между множеством событий и временной синхронизации активизации элементов IDEF3-диаграмм. Различают перекрестки для слияния (Fan-in Junction) и разветвления (Fan-out Junction) стрелок. Перекресток не может использоваться одновременно для слияния и для разветвления. При внесении перекрестка в диаграмму необходимо указать его тип. Тип перекрестка определяет логику и временные параметры отношений между элементами диаграммы. Все перекрестки на диаграмме нумеруются, каждый номер имеет префикс «J». Тип перекрестка обозначается внутри элемента: & - логический И; О – логический ИЛИ; Х – логический перекресток неэквивалентности. Стандарт IDEF3 предусматривает разделение перекрестков типа & и О на синхронные и асинхронные. Это разделение позволяет учитывать в диаграммах описания процессов синхронизацию времени активизации. Мето-

дология IDEF3 использует пять логических типов для моделирования возможных последствий действий в сценарии. Все следующие процессы должны быть запущены:

Окончание одной работы может служить сигналом к началу нескольких работ, или же одна работа для своего запуска может ожидать окончания нескольких работ (рис.22). Перекрестки используются для отображения логики взаимодействия стрелок при слиянии и разветвлении или для отображения множества событий, которые могут или должны быть завершены перед началом следующей работы. Различают перекрестки для слияния (Fan-in Junction) и разветвления (Fan-out Junction) стрелок. Перекресток не может использоваться одновременно для слияния и разветвления.

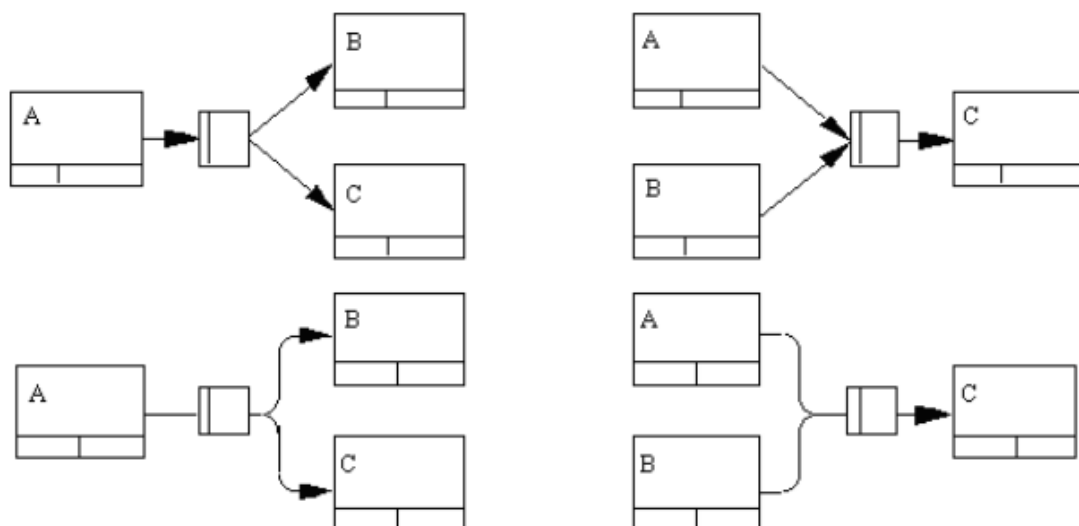


Рис.22. Перекрестки разветвления и слияния

Помимо наличия нескольких типов связей между работами в стандарте IDEF3 логические операторы, которые в данном случае называются перекрестками также делятся на несколько типов: "Исключающий ИЛИ", "И" и "ИЛИ".

Перекресток "Исключающий ИЛИ" обозначает, что после завершения работы "А" (рис. 23), начинает выполняться только одна из трех расположенных параллельно работ В, С или D в зависимости от условий 1, 2 и 3. Перекресток "И" обозначает, что после завершения работы "А", начинают

выполняться одновременно три параллельно расположенные работы В, С и D. Перекресток "ИЛИ" обозначает, что после завершения работы "А", может запуститься любая комбинация трех параллельно расположенных работ В, С и D. Например может запуститься только одна из них, могут запуститься три работы, а также могут запуститься двойные комбинации В и С, либо С и D, либо В и D. Перекресток "Исключающий ИЛИ" является самым неопределенным, так как предполагает несколько возможных сценариев реализации бизнес-процесса и применяется для описания слабо формализованных ситуаций.

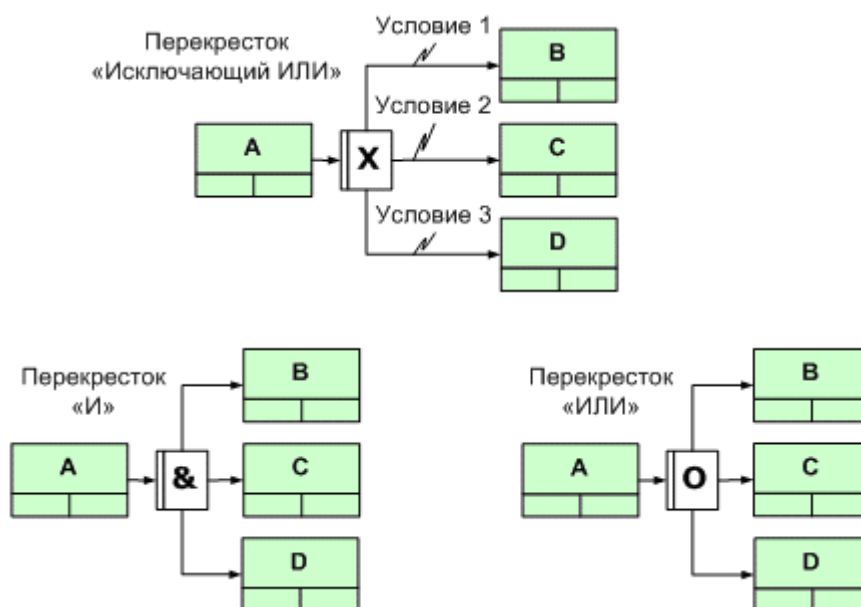


Рис.23. Применение перекрестков "Исключающий ИЛИ", "И" и "ИЛИ" - схемы расхождения.

Перекрестки "И" и "ИЛИ" подразделяются еще на два подтипа - синхронные и асинхронные. Перекрестки синхронного типа обозначают, что работы В, С и D запускаются одновременно после завершения работы А. Перекрестки асинхронного типа требований к одновременности не предъявляют. Приведенные на рис.4 схемы взаимосвязи работ и перекрестков называются схемами расхождения, так как от перекрестков расходятся не-

сколько работ. Существует и другие схемы взаимосвязи перекрестков и работ - это так называемые схемы схождения, когда к перекрестку подходит несколько работ (рис.24).

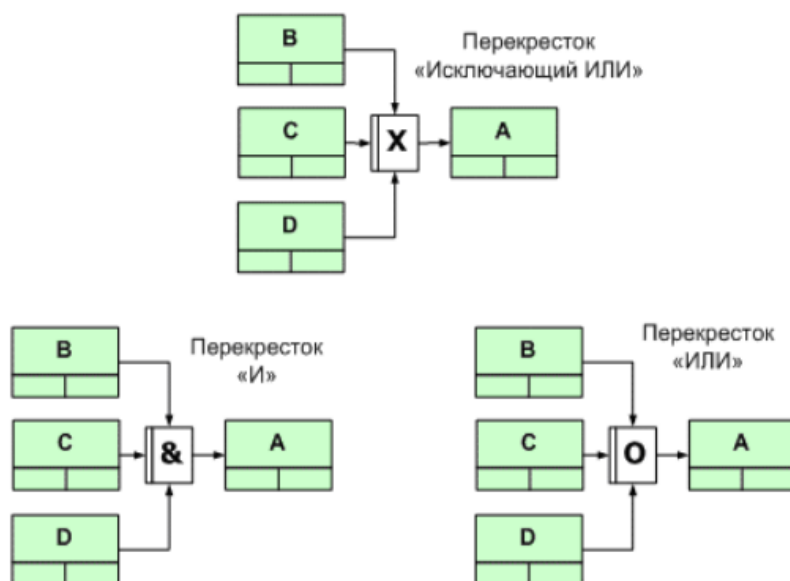


Рис.24. Применение перекрестков "Исключающий ИЛИ", "И" и "ИЛИ" - схемы схождения.

Правила использования перекрестков

1. Каждому перекрестку для слияния должен предшествовать перекресток для разветвления.
2. Перекресток «И» не может следовать за перекрестком для разветвления типа синхронного или асинхронного «О».
3. Перекресток «И» не может следовать за перекрестком типа исключающее «ИЛИ».
4. Перекресток исключающее «ИЛИ» не может следовать за перекрестком для разветвления типа «И».
5. Перекресток, имеющий одну стрелку на одной стороне не имеет более одной стрелки на другой стороне.

Объект ссылки в IDEF3 выражает некую идею, концепцию или данные, которые нельзя связать со стрелкой, перекрестком или работой.

Для внесения объекта ссылки служит кнопка «Добавить в диаграмму объект ссылки» (Referent) в палитре инструментов.

Объект ссылки изображается в виде прямоугольника, похожего на прямоугольник работы.

Имя объекта ссылки задается в диалоге Referent (пункт Name контекстного меню). В качестве имени можно использовать имя какой-либо стрелки с других диаграмм или имя сущности из модели данных.

Объекты ссылки должны быть связаны с единицами работ или перекрестками пунктирными линиями.

Официальная спецификация IDEF3 различает три стиля объектов ссылок:

- безусловные (unconditional);
- синхронные (synchronous);
- асинхронные (asynchronous).

Необходимо одну из работ, находящихся на диаграммах IDEF0, рассмотреть детально с помощью методологии IDEF3. При декомпозиции работы IDEF0 нужно учитывать, что стрелки на диаграммах IDEF0 означают потоки информации или объектов, передаваемых от одной работы к другой. На диаграммах IDEF3 стрелки могут показывать только последовательность выполнения работ, т.е. они имеют другой смысл, чем стрелки IDEF0. Поэтому при декомпозиции работы IDEF0 в диаграмму IDEF3 стрелки не мигрируют на нижний уровень. Если необходимо показать на дочерней диаграмме IDEF3 те же объекты, что и на родительских диаграммах IDEF0, необходимо использовать объекты ссылки.

При создании дочерней диаграммы BPWin переносит граничные стрелки родительской работы, их необходимо удалить и заменить на объекты ссылок. Заменяем стрелки «Заказы на настольные компьютеры», «Необходимые комплектующие», «Список необходимых комплектующих»,

«Настольные компьютеры» и «Результаты сборки» на объекты ссылок - кнопка «Referent» на панели инструментов, в появившемся окне выбрать переключатель «Arrow» и выбрать из списка нужное название.

Далее начинаем располагать на диаграмме работы, отражающие указанные выше действия, выполняемые при сборке компьютеров.

Примеры:

Пример 1. Сборка компьютера (рис. 25)

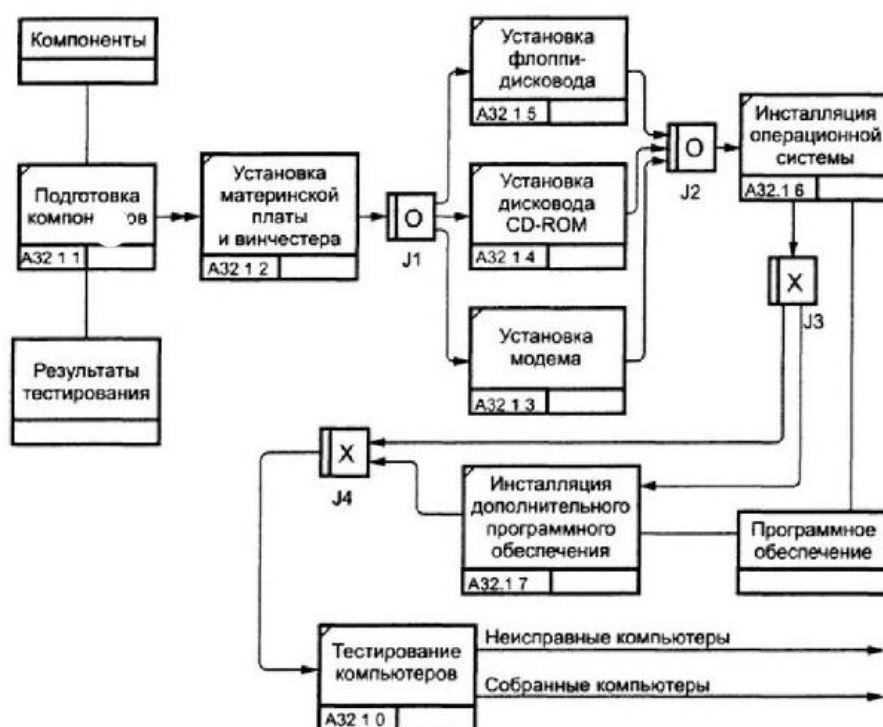
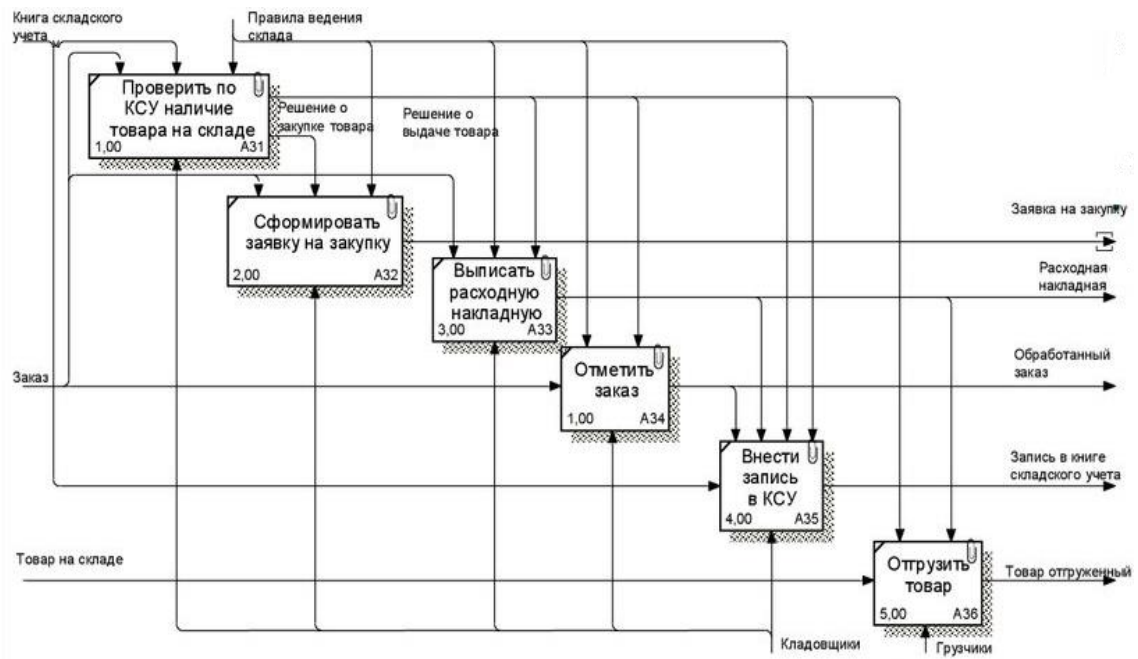
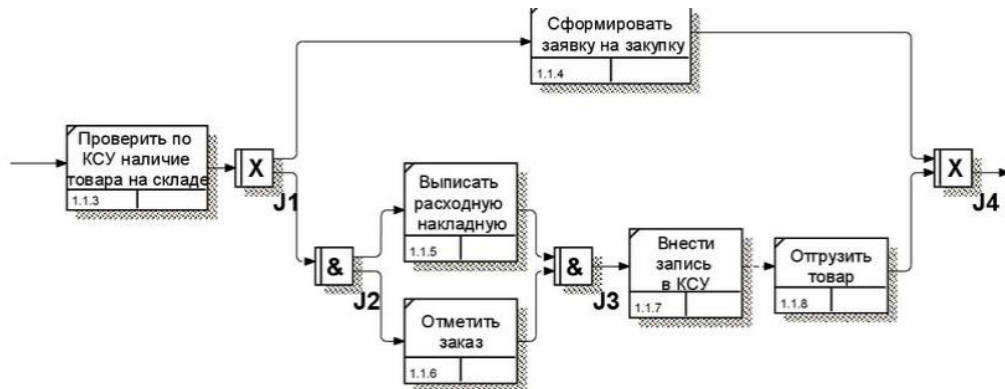


Рис.25. IDF3 диаграмма «Сборка компьютера».

Пример 2. Закупка товара (рис. 26 а-б).



а)



б)

Рис. 26. а) декомпозиция первого уровня IDF0; б) дополнение к IDF0
диаграмма IDF3

Пример 3. Выполнение курсовой работы (рис.27).

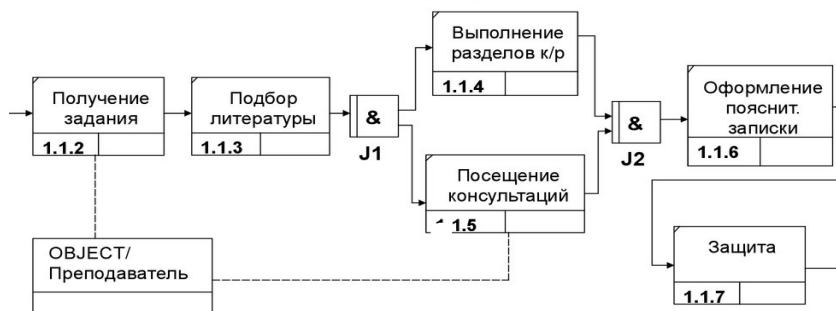


Рис.27. IDF3 диаграмма «Выполнение курсовой работы».

Пример 4. Продажа товаров (рис. 28).

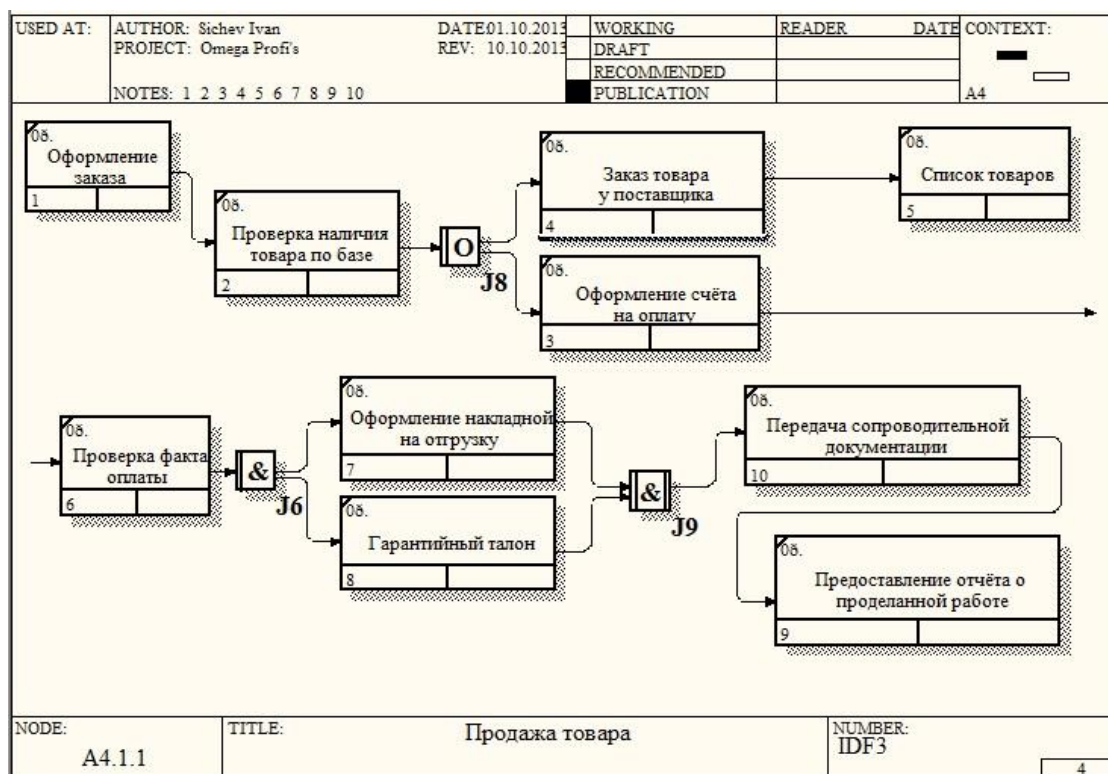


Рис.28. IDF3 диаграмма процесса «Продажа товара».

Пример 5. Обработка заказа (рис. 29).

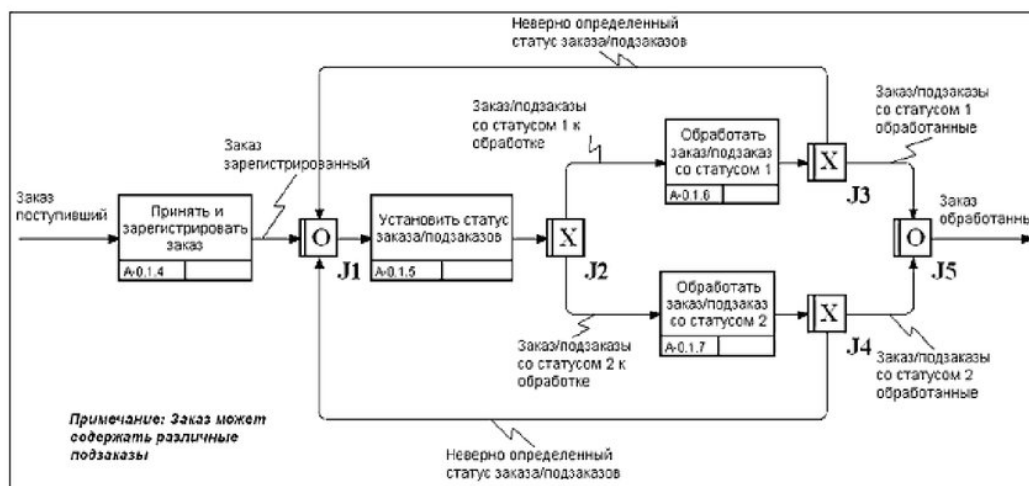


Рис.29. IDF3 диаграмма процесса «Обработка заказа».

Пример 6. Процесс оплаты товара (рис.30).

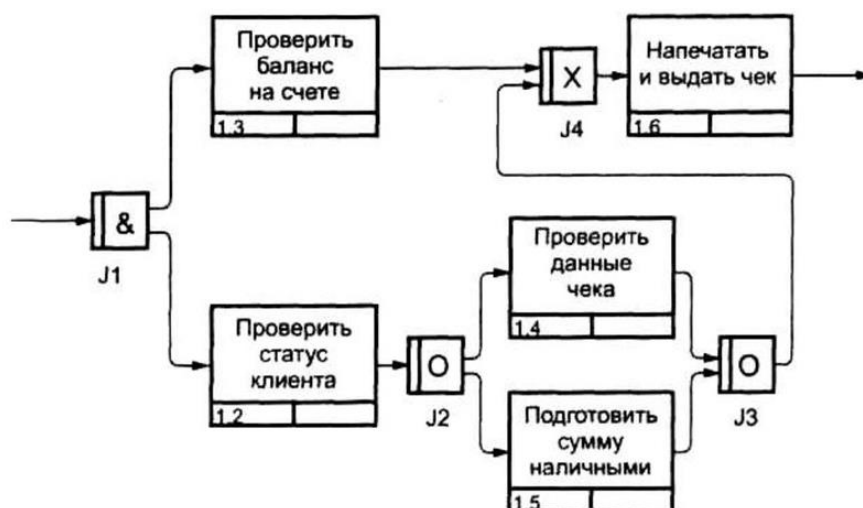


Рис.30. IDF3 диаграмма процесса «Оплата товара»

Пример 7. Процесс «Отбор слушателей с низким/ критическим уровнем посещаемости» (рис.31).

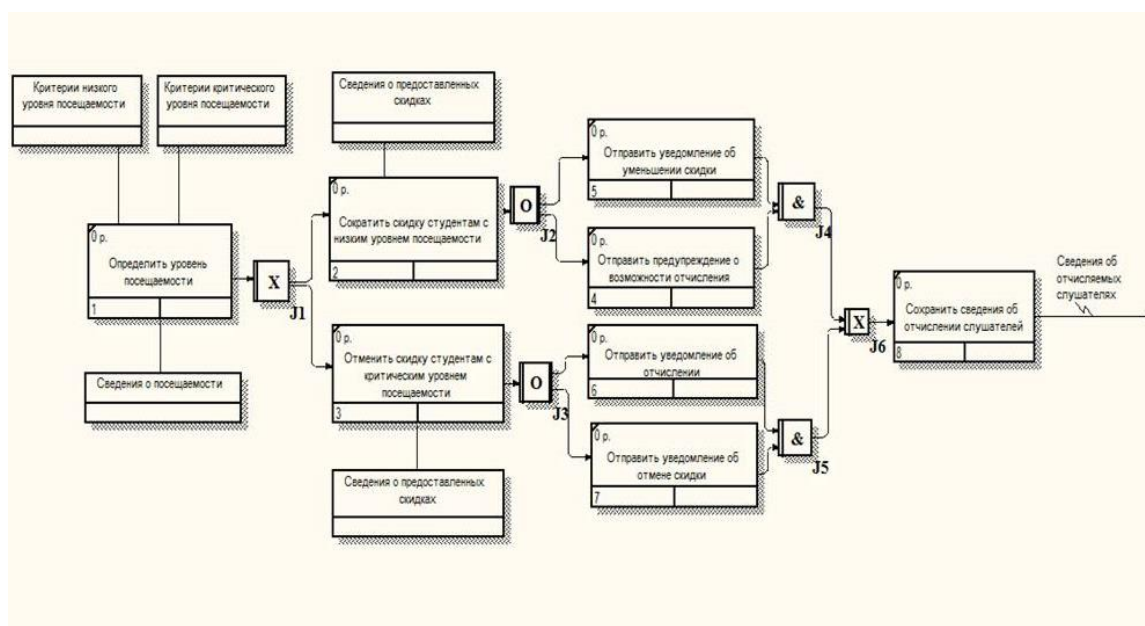


Рис.31. IDF3 диаграмма процесса «Отбор слушателей с низким/ критическим уровнем посещаемости»

2.3 Методология DFD

Целью методологии является построение модели рассматриваемой системы в виде диаграммы потоков данных (Data Flow Diagram – DFD). Диаграммы потоков данных предназначены прежде всего для описания документооборота и обработки информации, хотя допускают и представление других объектов.

DFD диаграммы в отличие от других нотаций позволяют визуально показать все процессы с точки зрения данных. Это может быть полезно:

- при разработке информационной системы;
- при интеграции системы;
- при миграции данных и функциональности с одной системы на другую;
- в проектах, связанных с Data Management;
- в процессе построения аналитического хранилища, BI-решения.

Диаграмма позволяет визуализировать как движение данных между объектами системы, так и преобразования данных, которые могут применяться на разных шагах процесса.

Элементы DFD диаграммы

Выделяют 4 элемента в диаграмме:

1. Процесс.

Процессы, при которых идет изменение потока данных (обработка, трансформация и др. изменения). Процесс как и в других диаграммах обычно прописывается с помощью глагола, например: “Отправка заполненной формы”.

2. Внешняя сущность.

Сущность (объект), которая получает или отправляет данные при взаимодействии с описанным процессом.

3. Хранилище данных.

Все хранилища данных или отдельные файлы, которые хранят исходные или выходные данные, а также все промежуточные хранилища.

4. Поток данных.

Поток данных, который отображает направление и сами данные, которые перемещаются между внешними сущностями и хранилищами данных с помощью процессов.

Несколько правил построения диаграмм:

- процесс должен иметь входной и выходной поток данных;
- хранилища данных также должны иметь входные и выходные потоки данных;
- данные с внешних сущностей должны обязательно проходить через процесс чтобы попасть в хранилище.

В DFD диаграммах также выделяют 2 разные нотации. Поэтому стоит обращать внимание на условные обозначения каждого элемента в зависимости от используемой нотации. Ниже представил картинку сравнения элементов разных нотаций (рис.32).

Уровни DFD Диаграммы

В зависимости от цели использования диаграммы можно отображать различные уровни детализации процесса. К примеру, для разговора и презентации процесса бизнес-пользователям и заказчикам, им важно понимать контекст и логику самого процесса, иногда нет смысла погружать их в технические моменты реализации. С другой стороны, при разговоре с технической командой важно сделать акцент на реализации решения с технической точки зрения.

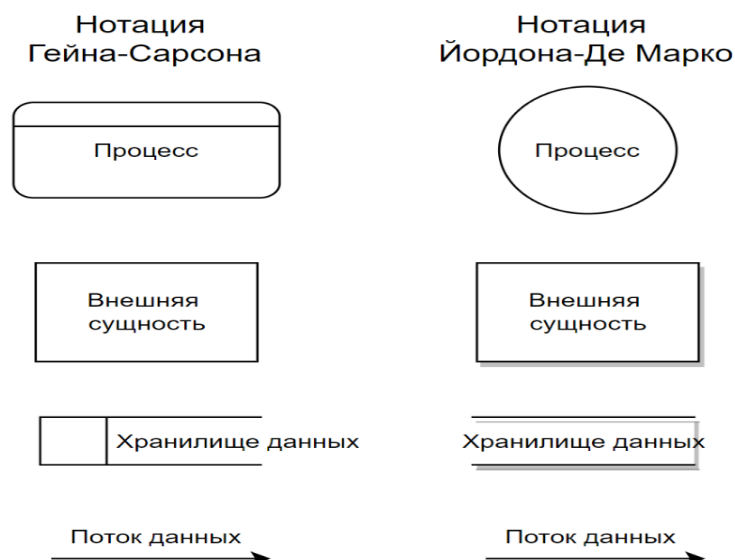


Рис.32. Элементов DFD в разных нотациях

Как и в ER-диаграмме для моделей данных, которая включает в себя несколько слоев отображения (концептуальный, логический, физический), DFD диаграммы также можно делить на подобные уровни:

1. **Концептуальный (или контекстный) уровень.** Показывает общее описание процесса, который реализуется при потоке данных. Отображает абстрактно потоки данных, связанные с разными внешними сущностями.
2. **Логический уровень (рис. 33).** Отображает логику преобразования данных в системе в каждом процессе, описывает. Видны входные, промежуточные, выходные данные в каждом процессе, который протекает от внешней сущности до хранилищ данных. Больше указывает на вопрос “Что включает в себя процесс потока и обмена данными со стороны бизнеса?”



Рис. 33. Логический уровень модели.

3. Физический уровень (рис. 34). Включают точное отображение хранилищ данных, названий сущностей данных. Диаграмма физического уровня должна отвечать на вопрос “Как будет реализован процесс передачи и потока данных?”

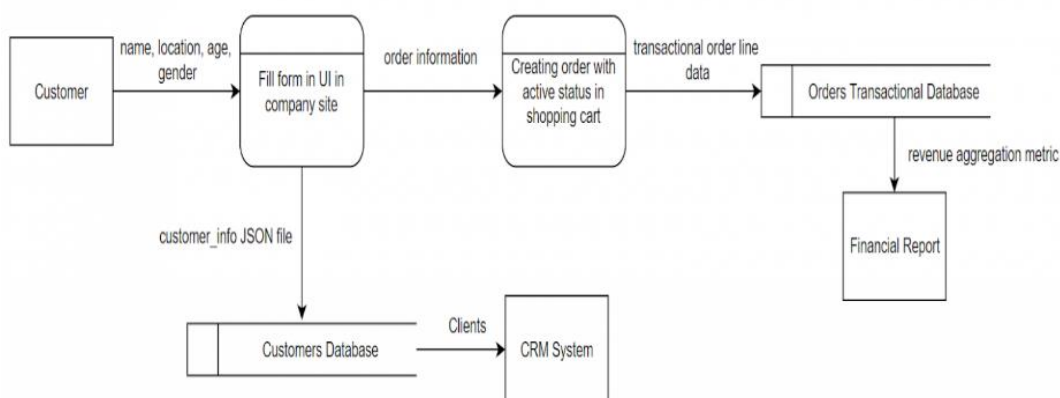


Рис.34. Физический уровень модели.

Также часто в других источниках можно увидеть разделение уровней диаграммы на 0,1, 2, 3 и так далее, в зависимости от уровня детализации.

Если мы говорим про разработку нового решения, то важно понять “что мы имеем сейчас” (AS-IS) и “что мы желаем получить” (TO-BE). Другими словами, мы разделяем наше текущее состояние и желаемое состояние, которое мы хотим получить с помощью нашего решения.

- **AS-IS** - описываем текущую логическую диаграмму;

- **ТО-ВЕ** - описываем желаемую логическую диаграмму с новой логикой и требованиями от бизнеса. После этого из желаемой логической диаграммы описываем физическую с новым техническим решением.

Примеры:

Пример 1. Контекстная DFD диаграмма «Запись на прием к врачу» (рис.35) и ее декомпозиция (рис. 36).



Рис. 35. Контекстная DFD диаграмма «Запись на прием к врачу»

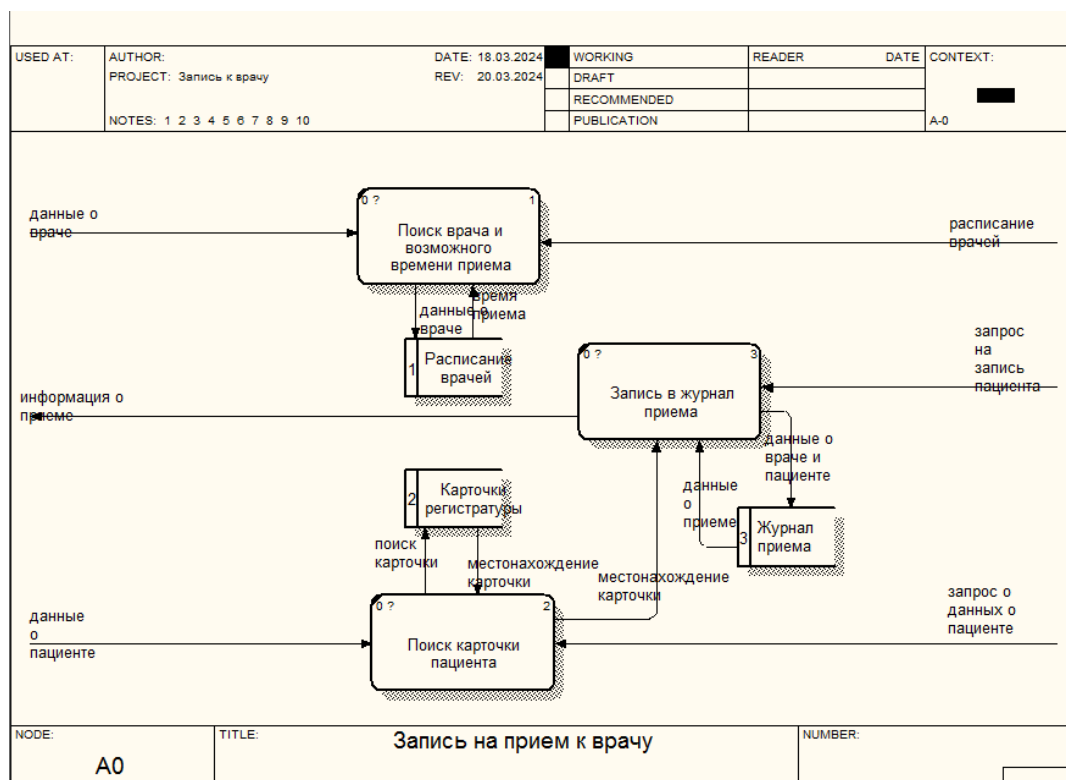


Рис.36. DFD декомпозиция «Запись на прием к врачу»

Пример 2: Контекстная DFD диаграмма Получение товара со склада (рис.37) ее декомпозиция работы предприятия со складом (ТО-ВЕ) (рис.38).



Рис.37. Диаграмма DFD процесса «Получение товара со склада».

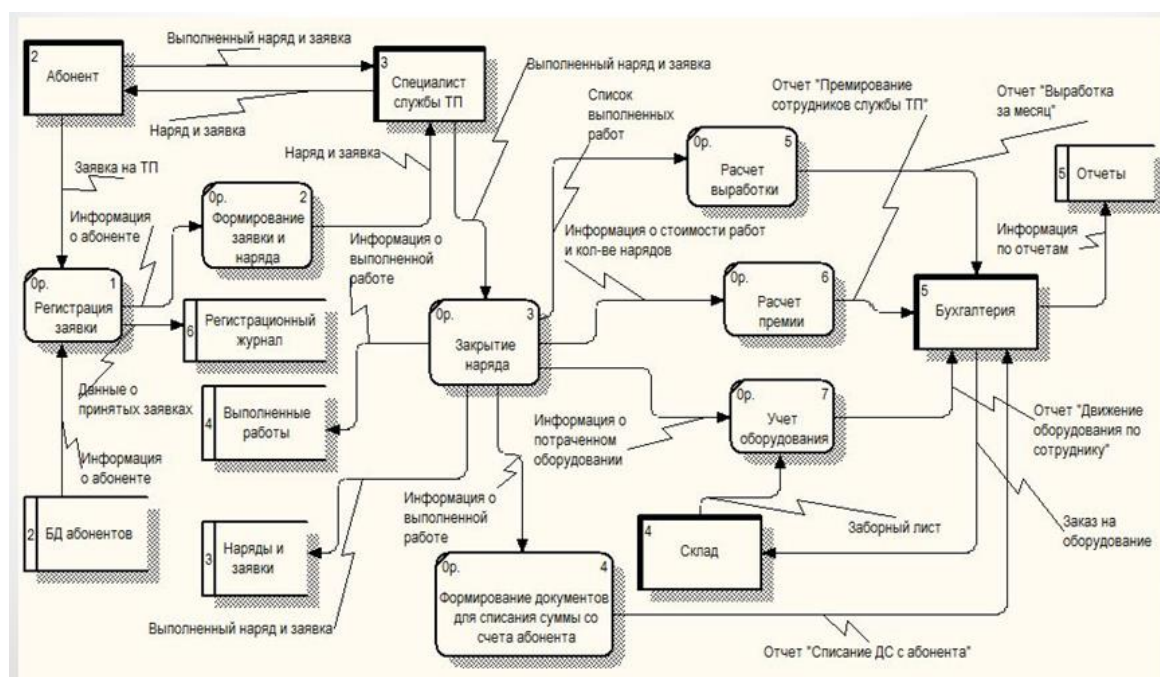


Рис.38. DFD диаграмма потока данных ТО ВЕ - уровень декомпозиции работы предприятия со складом

Пример 3: Контексная диаграмма процесса «Обучение в университете» (рис.39) и ее физическая модель (рис.40).

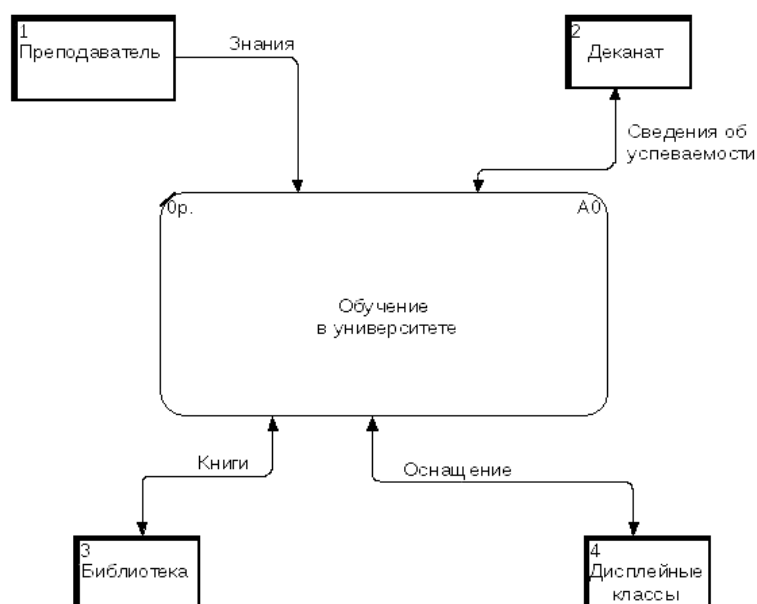


Рис.39. Контексная диаграмма DFD «Обучение в университете».

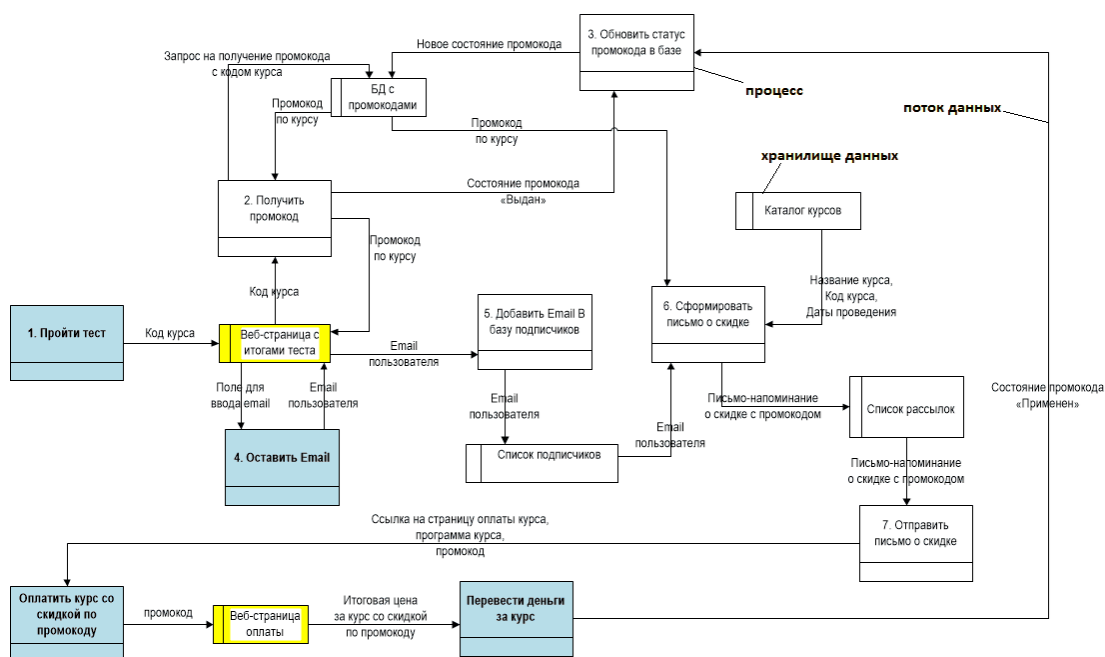


Рис.40. Физическая модель обучения в университете.

Задача. Медицинский центр

Условие задачи

В информационной системе присутствует имя, пол, дата рождения и домашний адрес каждого их пациента. Всякий раз, когда врач осматривает больного, явившегося к нему на прием, или сам приходит к нему на дом, он записывает дату и место, где проводится осмотр, симптомы, диагноз и предписания больному, проставляет имя пациента, а также свое имя. Если врач прописывает больному какое-либо лекарство, заносится название лекарства, способ его приема, словесное описание предполагаемого действия и возможных побочных эффектов. Спроектировать систему, которая позволяет:

- по заданной дате определить количество вызовов в этот день;
- позволяет определить количество заболевших данной болезнью;
- по заданному лекарству определить его побочный эффект;
- предоставить возможность добавления нового лекарства с описанием его свойств.

Контекстная диаграмма IDF0 представлена на рис.41 и ее последующие декомпозиции на рис.42 -49.

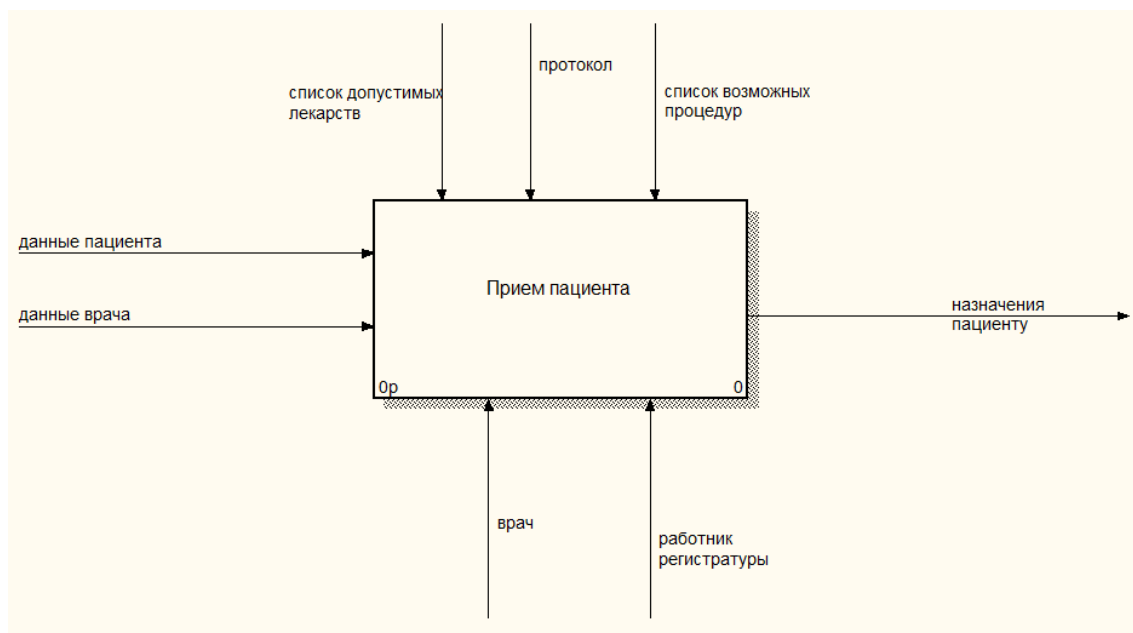


Рис. 41. Контексная диаграмма IDEF0

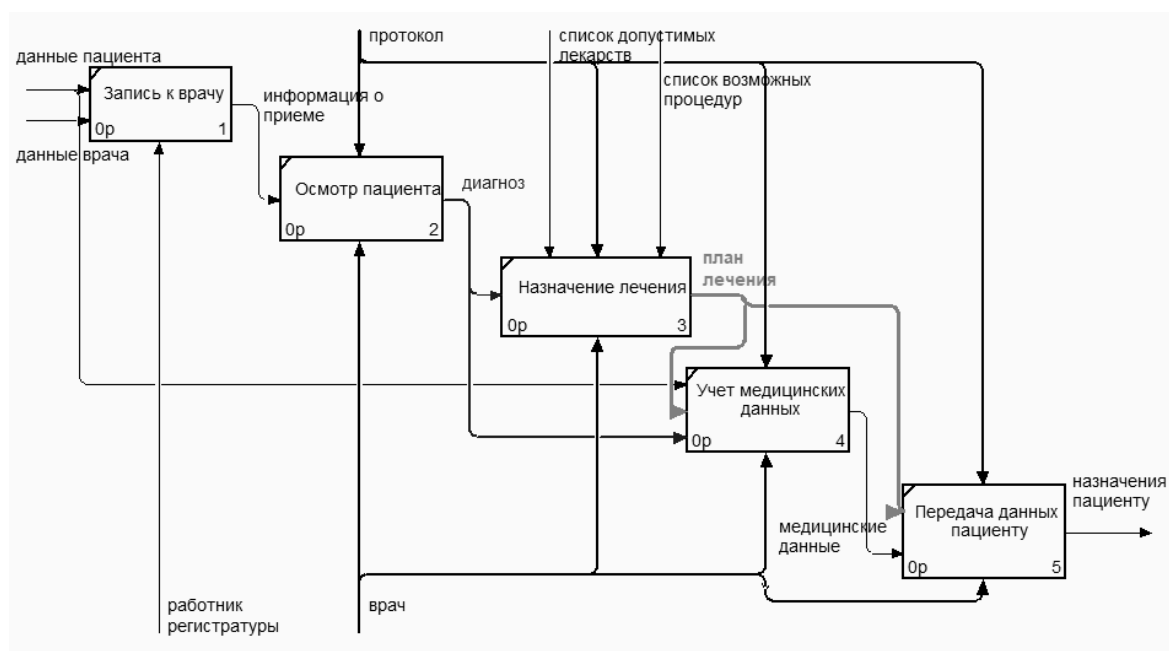


Рис.42. Первая декомпозиция IDEF0

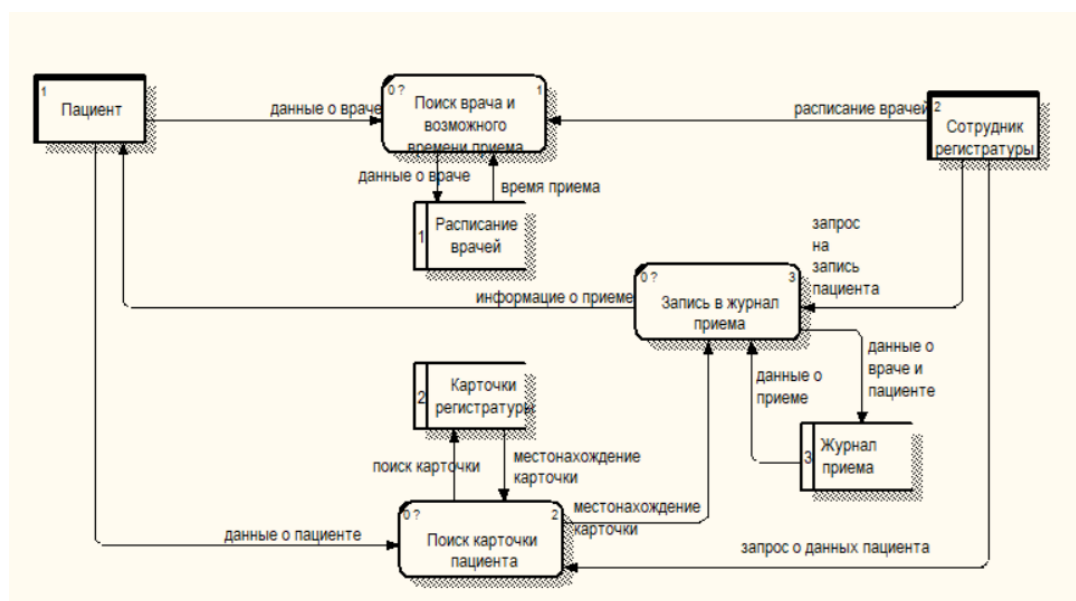


Рис.43. Диаграмма DFD-декомпозиция деятельности «Запись к врачу»

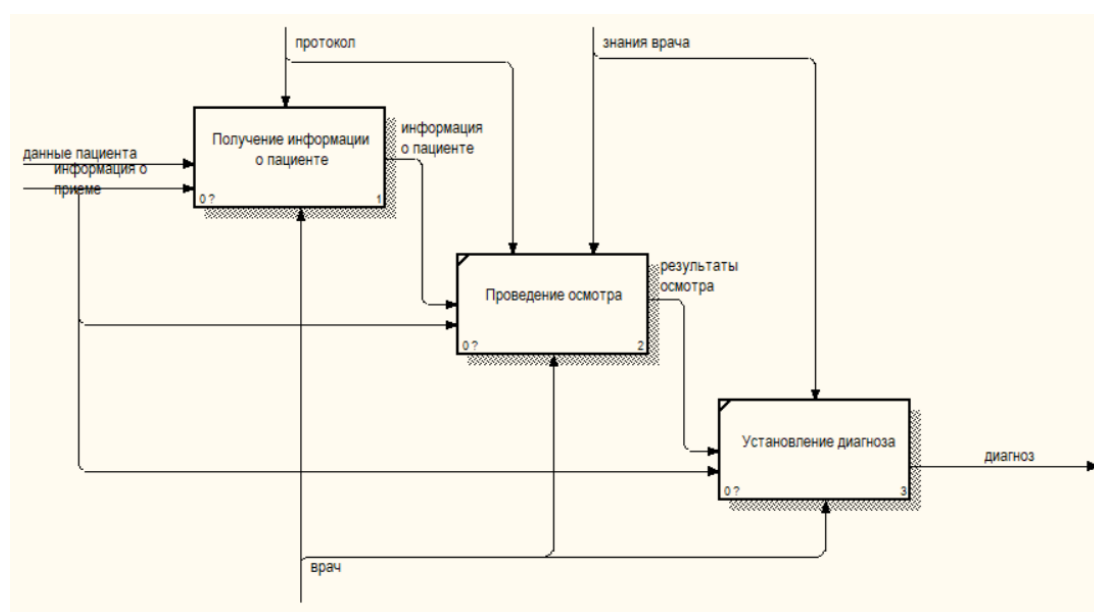


Рис.44. Декомпозиция IDEF0 деятельности «Осмотр пациента»

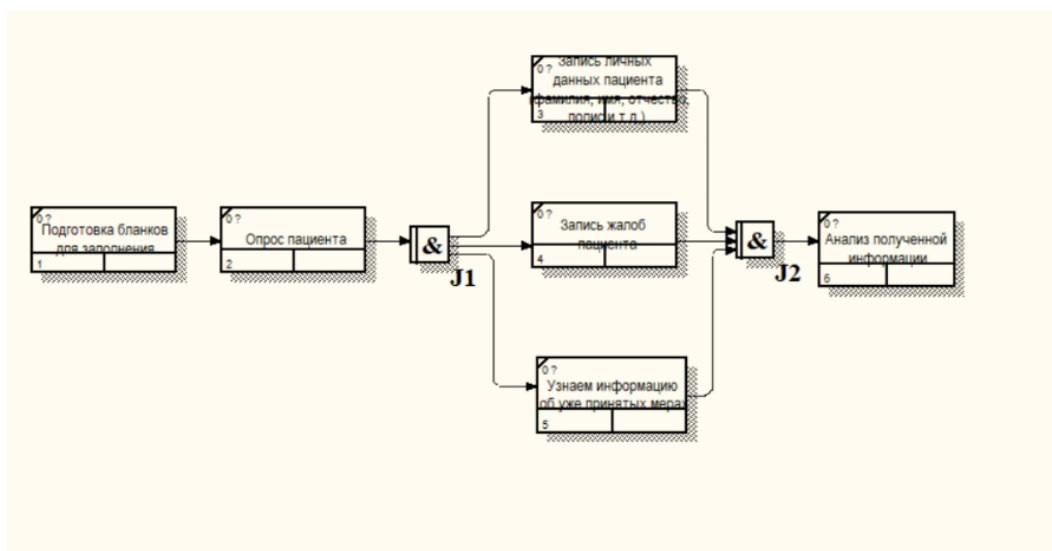


Рис.45. Диаграмма IDEF3 «Осмотр врача» и «Постановка диагноза»

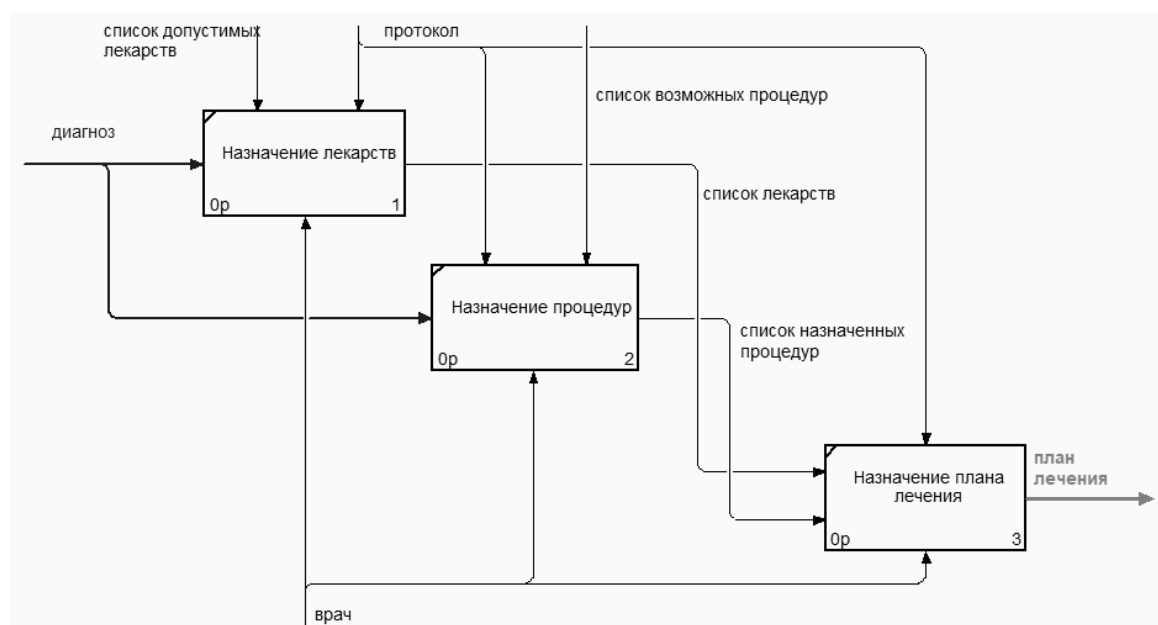


Рис.46. Декомпозиция деятельности «Назначение лечения».

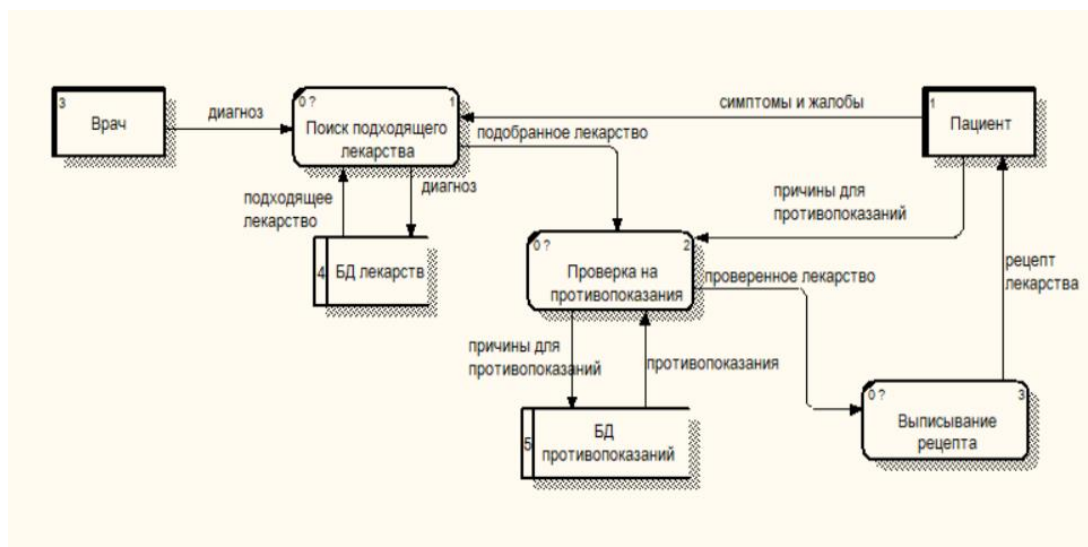


Рис.47. Декомпозиций DFD «Назначение лечения»

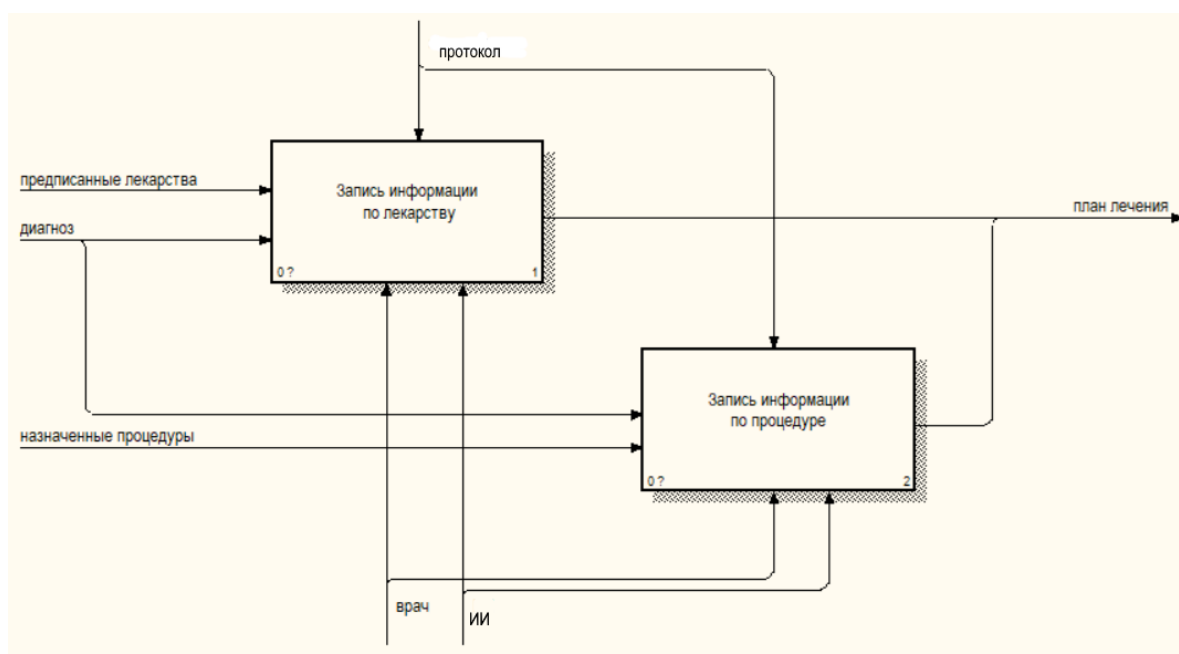


Рис.48. Декомпозиция деятельности «Назначение лекарства».

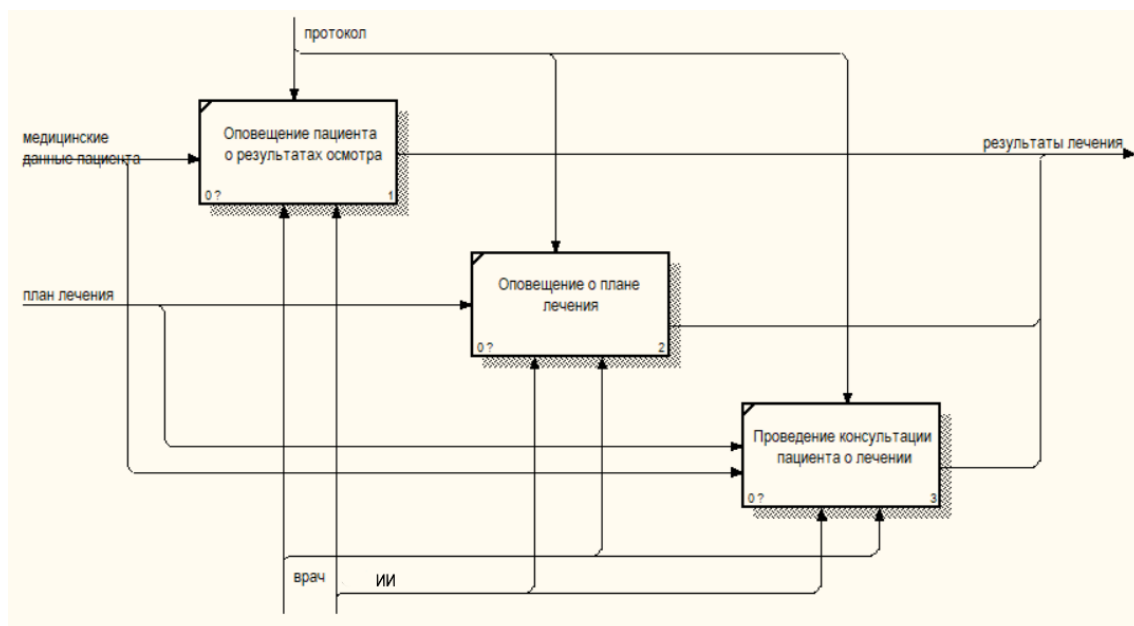


Рис.49. Декомпозиция деятельности «Назначения плана лечения».

Задания для самостоятельной работы

Все допущения к условиям задачи, которые не описаны в условиях должны быть описаны самостоятельно студентом. У каждого студента могут быть собственные допущения, которые будут влиять на ход решения задачи, то есть каждая диаграмма должна сопровождаться собственным описанием допущений к ней. В каждой задаче необходим как минимум 1 уровень декомпозиции. На оценку 5 необходимо сделать декомпозицию двух уровней.

1. Разработайте модель предметной области формирования заказа материалов на производственном предприятии, вместе с документом «Затраты предприятия». Менеджер по закупкам определяет вид и количество необходимых материалов, формирует заявку на них, отправляет заявку поставщику. Заполняет внутренний документ «Затраты на материалы», с которым автоматически считается сумма затрат на основании количества и цены материалов. Все заявки менеджеров 1 поставщику (за 1 день) объединяются в заказ. Все заказы и отчеты хранятся в базе данных. Пользователи менеджеры по закупкам, бухгалтер.

2. Разработайте модель предметной области для разработки информационной системы обучающей игры для детей. Основным участником игры – ребенок, ему помогает родитель. Ребёнок при помощи родителя (затем самостоятельно) выбирает одну из трех вариантов игры (1 вариант - изучение цифр, 2 вариант игры – изучение овощей и фруктов, 3 вариант игры – изучение животных). Внутри каждого варианта предусмотрено 2 уровня. Ко второму уровню можно перейти после успешного прохождения 1 уровня. Первый уровень ребенок может повторить при желании сколько угодно раз. После каждой попытки появляются результаты игры. Результаты и попытки сохраняются в базе данных. Пользователи: ребенок и родитель.

3. Разработайте модель предметной области для информационной системы прима заказа на доставку продуктов. Она должна содержать информацию о пользователях, заказах, продуктах, их количестве. Также – о курьерах, которые доставляют заказы (время и стоимость доставки для организации). Система должна формировать отчет о закупках на 1 смену. Отчет о доходах за смену, который можно фильтровать по курьерам, продуктам и заказчикам. Вся эта информация хранится в базе данных. Пользователи системы: менеджеры по закупкам товаров, бухгалтер, курьеры.

4. Разработайте модель предметной области для информационной системы выбора тура для туристического оператора. В системе можно будет выбрать место назначения, отель, тип питания, тип размещения. Система позволит бронировать туры и оплачивать их несколькими способами оплаты. Туроператор имеет возможность сформировать отчеты по пользователям, названию туров, стоимости. В системе формируется внутренний отчет по доходам за определенный период. Пользователи – менеджеры по продажам туроператора, бухгалтер.

5. Разработайте модель предметной области для информационной системы логистики бизнеса – транспортной компании, которая формирует маршруты и выбирает вид транспорта в зависимости от заявок на перевозку грузов. В системе учитываются грузы, их характеристики (объем, вес), заказчики, пункт назначения. Также система включает имеющийся транспорт, его статус (свободен, загружается, в пути), дает возможность рассчитать стоимость перевозки в зависимости от расстояния и груза. Выбрать маршрут. Пользователи: менеджеры компании, частично – заказчики.

ЛИТЕРАТУРА

1. Федеральный закон Российской Федерации от 27 июля 2006 г. N 149-ФЗ. Об информации, информационных технологиях и о защите информации.
2. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированных систем
3. ГОСТ 24.202-80. Требования к содержанию документа «Технико-экономическое обоснование создания АСУ. ГОСТ 34.201-89. Информационная технология. Комплекс стандартов на автоматизированные системы.
4. ГОСТ 34.201-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем .
5. Буч Г. Объектно ориентированный анализ и проектирование с примерами приложений на С++ / Г. Буч. – М.: Бином; СПб: Невский диалект, 1999. – 560 с.
6. ГОСТ Р ИСО 15288-2005 Информационная технология. Системная инженерия. Процессы жизненного цикла систем
7. Горбаченко, В.И. Проектирование информационных систем с СА ERwin Modeling Suite 7.3: учебное пособие / В. И. Горбаченко, Г. Ф. Убиенных, Г. В. Бобрышева – Пенза: Изд-во ПГУ, 2012. – 154 с.

У ч е б н о е и з д а н и е

**Астахова Ирина Федоровна,
Трофименко Елена Владимировна,
Хицкова Юлия Владимировна,
Горковенко Галина Валерьевна**

**ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННЫХ СИСТЕМ.
СТРУКТУРНОЕ ПРОЕКТИРОВАНИЕ**

Учебное пособие

Издано в авторской редакции

Подписано в печать 27.09.2024. Формат 60×84/16.
Усл. печ. л. 4,7. Тираж 50 экз. Заказ 407

Издательский дом ВГУ
394018 Воронеж, пл. Ленина, 10
Отпечатано с готового оригинал-макета
В типографии Издательского дома ВГУ
394018 Воронеж, ул. Пушкинская, 3